



Trabajo fin de grado
Doble Grado en Física e Ingeniería Electrónica
(Grado en Ingeniería Electrónica)

El derecho al olvido: técnicas de *Machine Unlearning* y su aplicación a redes neuronales profundas

Autor:
Roman Aseguinolaza Gallo
Director:
Luis Javier Rodríguez Fuentes

© 2024, Roman Aseguinolaza Gallo

Leioa, 20 de Junio de 2024

Resumen

Este trabajo aborda el derecho al olvido en el contexto de las redes neuronales profundas mediante la implementación de técnicas de *Machine Unlearning*. Se investigan y desarrollan técnicas de *Machine Unlearning* aplicadas a redes neuronales profundas, específicamente utilizando la arquitectura ResNet18 entrenada con el conjunto de datos CIFAR-10. El objetivo principal es verificar la capacidad de eliminar la influencia de un subconjunto de datos de entrenamiento, denominado “conjunto de olvido”, sin comprometer significativamente la precisión del modelo resultante. Para lograrlo, se implementan y evalúan dos enfoques principales: la fragmentación de datos y la manipulación del modelo a través de la reinicialización de capas y la depuración de pesos. Los resultados demuestran que, aunque es posible eliminar la influencia del conjunto de olvido, mantener la precisión original del modelo representa un desafío significativo.

Índice

Resumen	I
1. Introducción	1
1.1. Objetivos	2
2. Fundamento teórico	4
2.1. <i>Machine Learning</i>	4
2.1.1. Regresión logística	6
2.1.2. Redes neuronales	8
2.2. <i>Machine Unlearning</i>	10
2.2.1. Reorganización de datos	10
2.2.2. Manipulación del modelo	12
2.2.3. Verificación del olvido: <i>Membership Inference Attack</i>	12
3. Resultados y discusión	14
3.1. Entrenamiento y evaluación del modelo de estudio	14
3.1.1. Conjunto de datos: CIFAR-10	14
3.1.2. Modelo de estudio: ResNet18	14
3.1.3. Entrenamiento y evaluación de la precisión	15
3.1.4. Vulnerabilidad a un <i>Membership Inference Attack</i>	17
3.2. Fragmentación de datos	17
3.2.1. Entrenamiento aislado de los fragmentos	17
3.2.2. Entrenamiento no aislado de los fragmentos	19
3.3. Manipulación del modelo	20
3.3.1. Reinicialización de capas	20
3.3.2. Depuración de pesos	23
3.4. Depuración y fragmentación	27
4. Conclusiones	31
Referencias	32
A. Repositorio de Código	33

1. Introducción

El aprendizaje profundo ha impulsado recientemente un progreso tremendo en una amplia gama de aplicaciones, que van desde la generación de imágenes y sistemas de automatización en la industria impresionantes hasta modelos de lenguaje que pueden mantener conversaciones similares a las humanas. Aunque el progreso es espectacular, el uso generalizado de modelos de redes neuronales profundas requiere precaución: debemos desarrollar tecnologías de inteligencia artificial de manera responsable, entendiendo y mitigando los riesgos potenciales, como la propagación y amplificación de sesgos injustos y la protección de la privacidad del usuario.

Muchas aplicaciones del aprendizaje automático (ML, por sus siglas en inglés) implican el análisis de datos recopilados de individuos. Estos datos a menudo son de naturaleza sensible y podrían incluir información como registros médicos o correos electrónicos personales. Además, los flujos de datos a menudo no son estáticos: se recopilan nuevos datos regularmente y se utilizan de manera incremental para refinar los modelos existentes.

Por el contrario, también puede requerirse que los datos sean eliminados. Legislaciones introducidas recientemente, como el Reglamento General de Protección de Datos (GDPR) en la Unión Europea, la Ley de Privacidad del Consumidor de California en los Estados Unidos y la legislación de privacidad PIPEDA en Canadá, incluyen disposiciones que requieren el llamado *right to be forgotten*, o derecho a ser olvidado [1]. Este requisito, que ha sido uno de los más controvertidos en el GDPR, obliga a las empresas a tomar medidas razonables para lograr la eliminación de los datos personales de los individuos que lo requieran. La escala sin precedentes a la que se está aplicando el ML alimentado con datos personales obliga a examinar cómo este derecho a ser olvidado puede implementarse eficientemente en los sistemas de ML.

Borrar completamente la influencia de los datos que se solicita eliminar es un problema difícil de resolver, ya que, además de simplemente eliminarlos de las bases de datos donde se almacenan, también requiere borrar la influencia de esos datos en los modelos de aprendizaje automático entrenados. Además, investigaciones recientes han demostrado que, en algunos casos, puede ser posible detectar con alta precisión si un ejemplo se utilizó para entrenar un modelo de aprendizaje automático [2]. Esto puede generar una gran preocupación sobre la privacidad, ya que implica que, incluso si se eliminan los datos de un individuo de una base de datos, aún puede ser posible deducir si los datos de ese individuo se utilizaron para entrenar un modelo.

Borrar la influencia de ciertos datos en un modelo tiene aplicaciones más allá de la protección de la privacidad del usuario. Por ejemplo, se puede borrar información inexacta u obsoleta de los modelos entrenados (debido a errores en el etiquetado o cambios en el entorno) o eliminar datos dañinos, manipulados o atípicos.

Dado lo anterior, el *desaprendizaje automático* o *machine unlearning* es un subcampo emergente del aprendizaje automático que tiene como objetivo eliminar la influencia de un subconjunto específico de ejemplos de entrenamiento —el “conjunto de olvido”— de un modelo entrenado. Además, un algoritmo de *desaprendizaje* ideal eliminaría la influencia de ciertos ejemplos mientras mantiene otras propiedades beneficiosas, como la precisión en el resto del conjunto de entrenamiento y la generalización a otros ejemplos. Una forma

sencilla de producir este modelo desaprendido es re-entrenar el modelo en un conjunto de entrenamiento que excluya las muestras del conjunto de olvido. Sin embargo, esto no siempre es una opción viable, ya que re-entrenar modelos complejos puede suponer un gran coste computacional. Un algoritmo de *desaprendizaje* ideal, en cambio, usaría el modelo ya entrenado como punto de partida y realizaría ajustes eficientes para eliminar la influencia de los datos solicitados. Se muestra en la Figura 1.1 un esquema de evaluación de métodos de *desaprendizaje automático*.

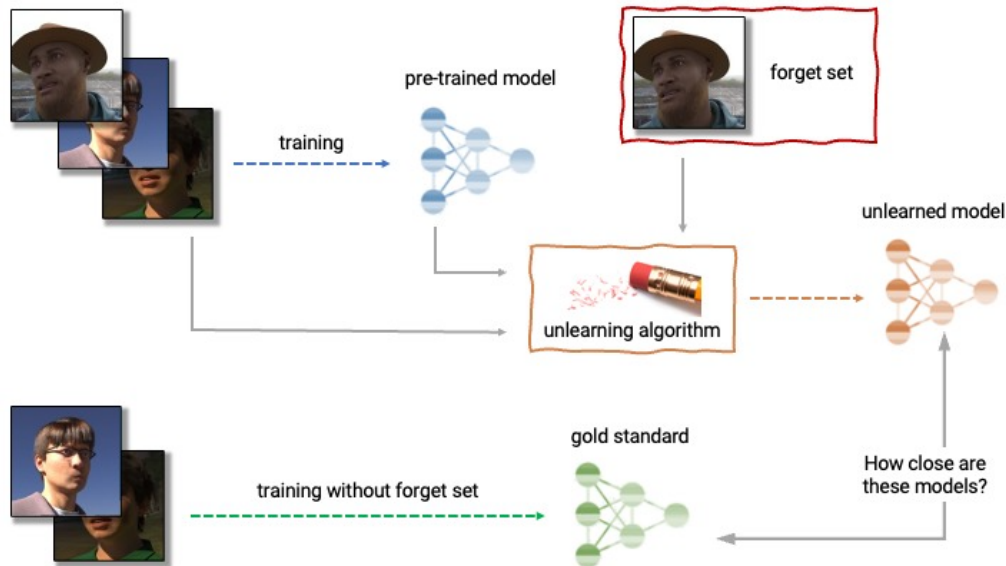


Figura 1.1: Esquema de *desaprendizaje*. Un algoritmo de *desaprendizaje* toma como entrada un modelo pre-entrenado y una o más muestras del conjunto de entrenamiento para desaprender (el “conjunto de olvido”). A partir del modelo, el conjunto de olvido y el conjunto retenido, el algoritmo de *desaprendizaje* produce un modelo actualizado. Un algoritmo de *desaprendizaje* ideal produce un modelo que es indistinguible del modelo entrenado sin el conjunto de olvido.

El campo del *desaprendizaje automático* está relacionado con otras áreas del aprendizaje automático, como la privacidad diferencial, el aprendizaje continuo y la equidad. La privacidad diferencial tiene como objetivo garantizar que ningún ejemplo de entrenamiento particular tenga una influencia demasiado grande en el modelo entrenado; un objetivo algo distinto en comparación con el *desaprendizaje*, que solo requiere borrar la influencia del conjunto de olvido designado. La investigación en aprendizaje continuo tiene como objetivo diseñar modelos que puedan aprender continuamente mientras mantienen las habilidades adquiridas previamente. A medida que avanza el trabajo en el *desaprendizaje*, también puede abrir formas adicionales de mejorar la equidad en los modelos, corrigiendo sesgos injustos o el tratamiento dispar de miembros pertenecientes a diferentes grupos (por ejemplo, grupos étnicos, grupos de edad, etc.).

1.1. Objetivos

El objetivo principal de este trabajo es desarrollar y evaluar técnicas de *Machine Unlearning* que aplicaremos a redes neuronales profundas, con el fin de implementar el derecho al olvido en estos sistemas. Los objetivos específicos son:

- Analizar las diferentes metodologías existentes para *Machine Unlearning*.
- Implementar y comparar dos estrategias de *desaprendizaje*: fragmentación de datos y manipulación del modelo.
- Evaluar el impacto de estas técnicas en la precisión y eficiencia del modelo, utilizando el conjunto de datos CIFAR-10 y la arquitectura ResNet18.
- Proponer una metodología obtenga un equilibrio óptimo entre la eliminación de datos y el rendimiento del modelo.

2. Fundamento teórico

2.1. *Machine Learning*

El *Machine Learning* (ML), o aprendizaje automático, es una rama de la inteligencia artificial que se centra en el desarrollo de algoritmos y técnicas que permiten a los ordenadores aprender a partir de datos. En lugar de ser programadas explícitamente para realizar una tarea específica, los modelos entrenados mediante algoritmos de ML utilizan grandes conjuntos de datos para identificar patrones y hacer predicciones o tomar decisiones informadas.

El aprendizaje automático puede dividirse en varias categorías, siendo las más destacadas el aprendizaje supervisado y el no supervisado. El aprendizaje supervisado se basa en un conjunto de datos etiquetados, lo que significa que cada entrada en el conjunto de datos de entrenamiento está asociada con una salida deseada conocida. El objetivo de un algoritmo de aprendizaje supervisado es aprender una función que mapee las entradas a las salidas correctas. Los dos tipos principales de tareas en el aprendizaje supervisado son la regresión y la clasificación.

La regresión es una tarea que implica predecir un valor continuo. Por ejemplo, un modelo de regresión puede predecir el precio de una casa basado en características como el tamaño, la ubicación y el número de habitaciones. La clasificación, por otro lado, implica predecir una categoría discreta. Un ejemplo clásico es un modelo que clasifica correos electrónicos como “spam” o “no spam”. Ambos tipos de tareas son esenciales en múltiples aplicaciones del mundo real y requieren diferentes tipos de algoritmos y enfoques.

Por otro lado, el aprendizaje no supervisado se ocupa de datos que no tienen etiquetas explícitas. El objetivo aquí es encontrar patrones o estructuras ocultas en los datos. Una técnica común en el aprendizaje no supervisado es el *clustering*, que agrupa datos en distintos conjuntos utilizando para ello medida de similitud. Un ejemplo de *clustering* sería agrupar clientes en segmentos de mercado similares basados en su comportamiento de compra.

La Figura 2.1 representa unos esquemas que reflejan cada una de las categorías mencionadas.

Aunque la mayoría de los datos de los que disponemos no están etiquetados, el aprendizaje supervisado es el más empleado e investigado hoy en día. Esto se debe a la eficacia y precisión que estos modelos pueden alcanzar al tener ejemplos específicos de entrada y salida para aprender. Para estudiar y desarrollar algoritmos de *machine unlearning*, nos centraremos en algoritmos aplicados a modelos entrenados con datos etiquetados.

Para entender cómo se entrenan y evalúan estos modelos, es fundamental introducir algunos conceptos clave, como el coste y la precisión, así como los datos de entrenamiento, prueba y validación.

El coste es una medida que indica cómo de bien se están ajustando las predicciones del modelo a los datos reales. Específicamente, una función de coste cuantifica el error entre las predicciones del modelo y los valores esperados. Por ejemplo, en la regresión lineal, se suele utilizar el error cuadrático medio (MSE, por sus siglas en inglés), que mide el promedio de los cuadrados de las diferencias entre las predicciones del modelo y los

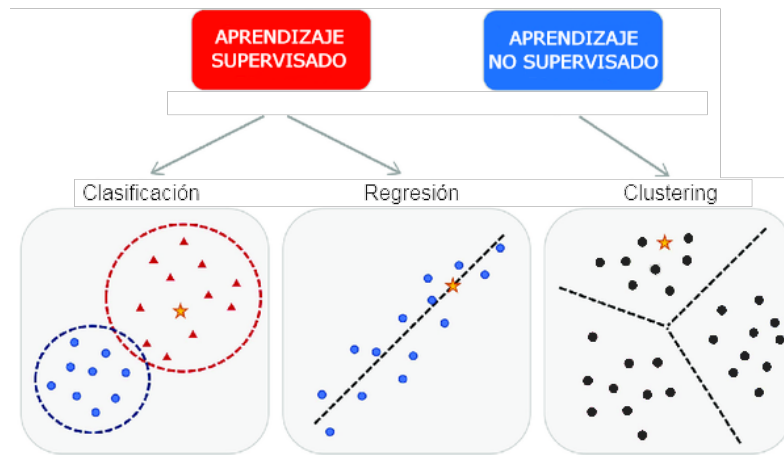


Figura 2.1: Ejemplos de los principales tipos de aprendizaje en ML: clasificación y regresión en aprendizaje supervisado, y *clustering* en aprendizaje no supervisado.

valores reales. El objetivo durante el entrenamiento es minimizar esta función de coste, ajustando los parámetros del modelo para reducir el error.

La precisión es una métrica que indica cómo de bien está funcionando el modelo. En tareas de clasificación, por ejemplo, la precisión se mide comúnmente como el porcentaje de predicciones correctas realizadas por el modelo sobre un conjunto de datos.

Para entrenar y evaluar un modelo de manera efectiva, los datos se dividen generalmente en tres conjuntos: entrenamiento, validación y prueba.

El conjunto de entrenamiento se utiliza para ajustar los parámetros del modelo, permitiéndole aprender los patrones en los datos. El sobreajuste es un problema crítico que queremos evitar cuando entrenamos el modelo con los datos de entrenamiento. Un modelo sobreajustado tiene un rendimiento excelente en el conjunto de entrenamiento pero falla al hacer predicciones precisas sobre nuevos datos. Para cerciorarnos de que no se están sobreajustando los parámetros del modelo a los datos de entrenamiento, separamos un conjunto de prueba y evaluamos la precisión del modelo en este conjunto. Si la precisión sobre los datos de prueba del modelo es tan buena como la precisión que se ha logrado sobre el conjunto de entrenamiento, el modelo es capaz de generalizar bien a datos que no ha visto.

Sin embargo, muchas veces no es suficiente con aislar un único conjunto de prueba, porque además de ajustar los parámetros del modelo durante el entrenamiento, hay que ajustar los hiperparámetros que configuran el comportamiento del modelo y del propio algoritmo de entrenamiento (arquitectura del modelo, tasa de aprendizaje, etc.). El conjunto de validación se emplea precisamente para ajustar los hiperparámetros y prevenir el sobreajuste del modelo a los datos de entrenamiento y de prueba.

A continuación vamos a discutir en más detalle los algoritmos de ML que van a aplicarse en este trabajo: la regresión logística y las redes neuronales. Ambos son algoritmos de aprendizaje supervisado, que pueden emplearse tanto para tareas de regresión como para tareas de clasificación.

2.1.1. Regresión logística

La regresión logística se emplea habitualmente para estimar la probabilidad de que un dato de entrada pertenezca a una clase particular. Puede definirse como clasificador binario si se considera que con una probabilidad por encima del 50 % predice que el dato pertenece a la clase, y que si está por debajo predice lo contrario.

De forma similar a una regresión lineal, la regresión logística calcula la probabilidad como la suma ponderada de las características de los datos de entrada, pero la probabilidad estimada por el modelo es el valor de la función logística de esa suma:

$$\hat{p} = h_{\theta}(\mathbf{x}) = \sigma(\mathbf{x}^T \boldsymbol{\theta}). \quad (1)$$

En la ecuación (1), $\boldsymbol{\theta}$ es el vector de parámetros del modelo, $\mathbf{x}$ es el vector de características de los datos de entrada, y h_{θ} se denomina función hipótesis, que en este caso se implementa con una función sigmoide $\sigma(\cdot)$ definida como sigue:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad (2)$$

representada en la Figura 2.2.

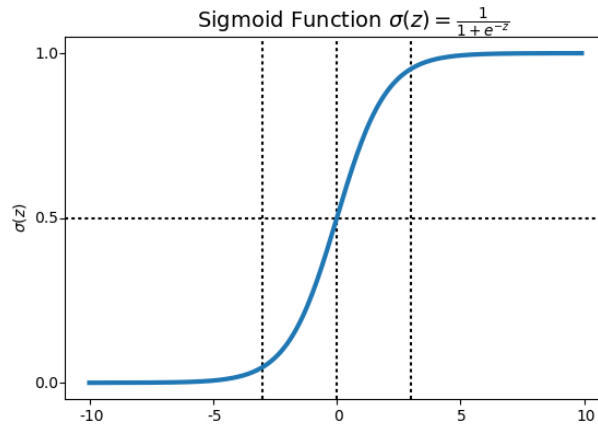


Figura 2.2: Comportamiento de la función $\sigma(z)$.

$\hat{p}$ da la probabilidad de pertenencia a una clase, y la predicción del modelo $\hat{y}$ para el dato de entrada $\mathbf{x}$ viene dada por:

$$\hat{y} = \begin{cases} 0 & \text{si } \hat{p} < 0,5, \\ 1 & \text{si } \hat{p} > 0,5. \end{cases} \quad (3)$$

Los parámetros del modelo $\boldsymbol{\theta}$ se determinan entrenando el modelo, de forma que se obtengan probabilidades altas para los datos de entrada que pertenecen a la clase ($y = 1$) y probabilidades bajas para los datos que no pertenecen a la clase ($y = 0$). Puede definirse, por tanto, la siguiente función de coste, que deberá minimizarse para el conjunto de datos de entrada $\mathbf{x}$ y sus pertenencias reales y :

$$c(\boldsymbol{\theta}) = \begin{cases} -\log \hat{p} & \text{si } y = 1, \\ -\log (1 - \hat{p}) & \text{si } y = 0. \end{cases} \quad (4)$$

Para $y = 1$, el coste es enorme cuando la probabilidad que predice el modelo tiende a 0, y se anula cuando tiende a 1. Si $y = 0$ ocurre lo contrario. Por tanto, minimizando esta función $c(\boldsymbol{\theta})$ lograremos un modelo muy preciso con respecto a la entrada $\mathbf{x}$. Si definimos una función de coste como la media de los costes sobre todos los datos de entrenamiento:

$$J(\boldsymbol{\theta}) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]. \quad (5)$$

Como ocurre con la mayoría de las funciones de coste que se emplean en algoritmos de ML, no hay una solución analítica para calcular directamente los parámetros $\boldsymbol{\theta}$ del modelo que minimicen $J(\boldsymbol{\theta})$. Por suerte, $J(\boldsymbol{\theta})$ es una función convexa, y se pueden utilizar algoritmos de optimización como el descenso por gradiente para hallar el mínimo.

El algoritmo de descenso por gradiente ajusta los parámetros de forma iterativa para minimizar la función de coste, partiendo de unos valores iniciales aleatorios para $\boldsymbol{\theta}$ (ver la Figura 2.3). Para ello, en cada iteración se calcula el gradiente de la función:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} J(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} J(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\boldsymbol{\theta}) \end{bmatrix}, \quad (6)$$

y se actualizan los parámetros como sigue:

$$\boldsymbol{\theta}^{k+1} = \boldsymbol{\theta}^k - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})|_{\boldsymbol{\theta}^k}. \quad (7)$$

En la ecuación (7), k es el número de la iteración y η es la tasa de aprendizaje. La tasa de aprendizaje es un hiperparámetro que controla el tamaño de los pasos que se dan en cada iteración hacia el mínimo. Una tasa demasiado baja puede ralentizar mucho la convergencia, mientras que una tasa demasiado elevada puede hacer incluso que el procedimiento diverja y no se alcance el mínimo.

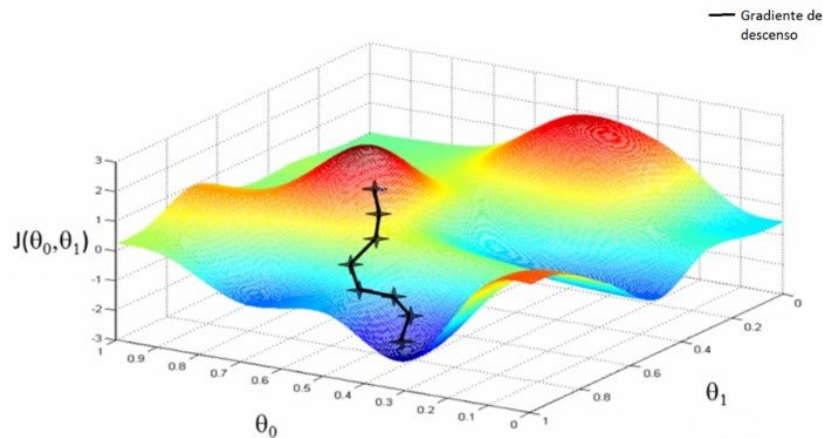


Figura 2.3: Proceso de aproximación al valor óptimo de los parámetros mediante descenso por gradiente.

2.1.2. Redes neuronales

Las redes neuronales son modelos de ML inspirados en la estructura y el funcionamiento del cerebro humano. Se componen de múltiples capas de neuronas artificiales que procesan la información y aprenden representaciones de datos en varios niveles de abstracción. Estos modelos pueden clasificarse en “shallow neural networks” (redes neuronales superficiales) y “deep neural networks” (DNNs o redes neuronales profundas), siendo las DNNs aquellas con un número significativo de capas. Actualmente, el término “redes neuronales” es prácticamente sinónimo de modelos de aprendizaje profundo.

Lo cierto es que una única neurona artificial se comporta de forma similar a la regresión logística. En la Figura 2.4.(a) se muestra el esquema de una neurona artificial con entradas $\mathbf{x}$, parámetros o pesos $\mathbf{w}$, un sesgo b y una función que se denomina de activación $f(\cdot)$. La neurona devuelve el valor y , dado por:

$$y = h_{\mathbf{w},b}(\mathbf{x}) = f(\mathbf{x}^T \mathbf{w} + b). \quad (8)$$

Notamos dos diferencias con la ecuación (1). La primera es que añadimos un sesgo b , y la segunda es que la función de activación $f(\cdot)$ no tiene porqué ser la función sigmoide. Es común emplear la función unidad rectificada uniforme (ReLU) definida como sigue:

$$f(z) = \text{ReLU}(z) = \max(0, z). \quad (9)$$

Por supuesto, una única neurona no presenta una enorme complejidad, pero su potencial emerge cuando conectamos las entradas y salidas de decenas, cientos o miles de neuronas dispuestas en capas (ver la Figura 2.4.(b)).

El proceso de aprendizaje de estos modelos consiste en actualizar los valores $\mathbf{w}$ que contienen todas las matrices de pesos, junto con los términos de sesgo de cada capa (en caso de haberlos) para ajustarse al resultado deseado.

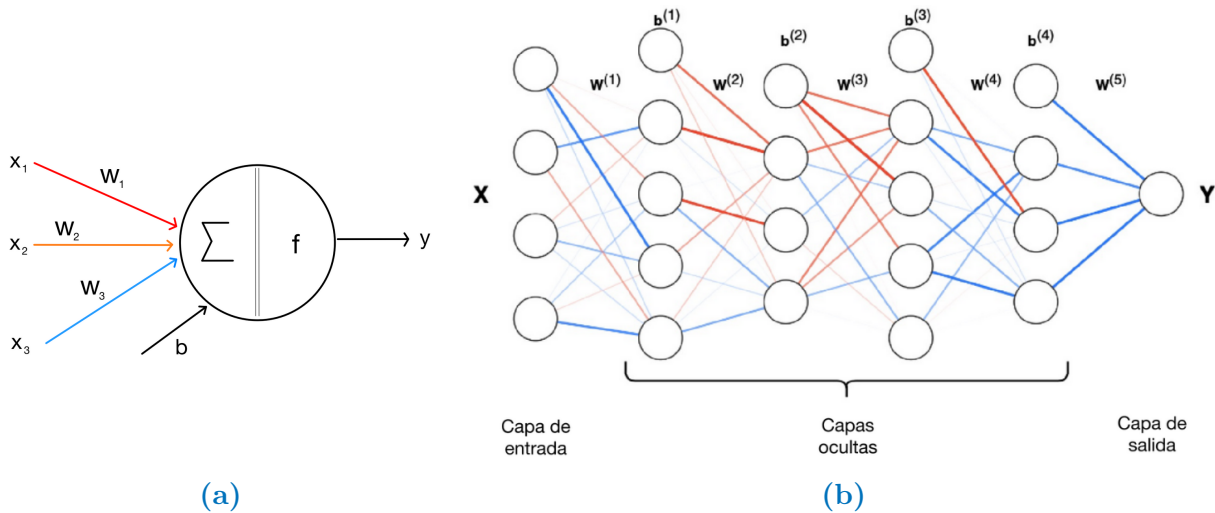


Figura 2.4: (a) Esquema de una neurona artificial. (b) Esquema de una red neuronal profunda con capas totalmente conectadas.

Las primeras neuronas artificiales fueron propuestas en las décadas de los 40 y los 50 [3, 4], y durante muchos años, los investigadores trataron sin éxito de encontrar una manera de entrenar múltiples capas de neuronas. Pero en 1986, David Rumelhart, Geoffrey Hinton y Ronald Williams publicaron un artículo revolucionario que introdujo el algoritmo de entrenamiento de propagación hacia atrás o retropropagación, que todavía se usa

hoy en día [5]. En resumen, se trata de un descenso por gradiente utilizando una técnica eficiente para calcular los gradientes automáticamente: en solo dos pasadas a través de la red (una hacia adelante y otra hacia atrás), el algoritmo de retropropagación es capaz de calcular el gradiente del coste de la red con respecto a cada parámetro del modelo. En otras palabras, puede determinar cómo se deben ajustar cada peso de conexión y cada término de sesgo para reducir el coste. Una vez que tiene estos gradientes, el algoritmo de retropropagación simplemente realiza un paso regular de descenso por gradiente, y todo el proceso se repite hasta que la red converge hacia la solución.

Estudiamos este algoritmo con más detalle:

- Maneja un solo lote a la vez (por ejemplo, con 32 datos de entrada o muestras), y pasa por todo el conjunto de entrenamiento múltiples veces. Cada pasada se llama una época.
- Cada lote se pasa a la capa de entrada de la red, que lo envía a la primera capa oculta. El algoritmo luego calcula la salida de todas las neuronas en esta capa (para cada muestra en el lote). El resultado se pasa a la siguiente capa, su salida se calcula y se pasa a la siguiente capa, y así sucesivamente hasta que obtenemos la salida de la última capa, la capa de salida. Este es la pasada hacia adelante: es exactamente como hacer predicciones, excepto que todos los resultados intermedios se preservan ya que se necesitan para la pasada hacia atrás.
- A continuación, el algoritmo mide el error de salida de la red, es decir, utiliza una función de coste que compara la salida deseada y la salida real de la red, y devuelve una medida del error.
- Luego calcula cuánto contribuyó cada conexión de salida al error. Esto se hace analíticamente aplicando la regla de la cadena, lo que hace que este paso sea rápido y preciso.
- A continuación, el algoritmo mide cuánto contribuye cada conexión en la capa inferior al error, nuevamente usando la regla de la cadena, yendo hacia atrás hasta que el algoritmo llega a la capa de entrada. Esta pasada hacia atrás mide eficientemente el gradiente de error a través de todos los pesos de conexión en la red al propagar este gradiente hacia atrás (de ahí el nombre del algoritmo).
- Finalmente, el algoritmo realiza un paso de descenso por gradiente para ajustar todos los pesos de conexión en la red, usando los gradientes de error que acaba de calcular.

Resumiendo, para cada lote de muestras de entrenamiento, el algoritmo de retropropagación primero hace una predicción (pasada hacia adelante) y mide el error, luego pasa por cada capa hacia atrás para medir la contribución del error de cada conexión, y finalmente ajusta los pesos de conexión para reducir el error (paso de descenso por gradiente).

Habiendo descrito el procedimiento de entrenamiento de las DNNs, es fácil comprender por qué es tan complicado conocer la influencia exacta de ciertos datos de entrenamiento en los pesos de la red. Además de la complejidad de la arquitectura de las redes profundas, en cada época en la que todas las muestras son empleadas por lotes para realizar el algoritmo de retropropagación, las muestras son divididas en lotes de manera aleatoria. Debido a la

aleatoriedad en el orden del entrenamiento, no existen algoritmos estadísticos eficientes para calcular la influencia de ciertas muestras en las DNNs [6].

2.2. *Machine Unlearning*

Una vez introducidos los fundamentos del *Machine Learning*, ya podemos definir qué es el *Machine Unlearning* (MU), cuales son los objetivos del MU y las distintas formas de abordar el problema.

Supongamos que tenemos un conjunto de datos de entrenamiento $\mathcal{D}$, y entrenamos un modelo M con sus parámetros $\mathbf{w}$ mediante un algoritmo de entrenamiento $\mathcal{A}(\cdot)$ (por ejemplo, un descenso por gradiente). Supongamos además que tenemos un subconjunto de los datos de entrenamiento $\mathcal{D}_u \subset \mathcal{D}$, cuya influencia en el modelo M queremos eliminar. $\mathcal{D}_u$ es el conjunto de olvido, mientras que el conjunto de retención es $\mathcal{D}_r = \mathcal{D} - \mathcal{D}_u$.

El algoritmo de *unlearning*, $\mathcal{U}(\mathcal{A}(\mathcal{D}), \mathcal{D}_r, \mathcal{D}_u)$, se define como una función que toma el modelo $\mathcal{A}(\mathcal{D})$ con parámetros $\mathbf{w}$, los conjuntos $\mathcal{D}_r$ y $\mathcal{D}_u$, y devuelve un modelo con parámetros $\mathbf{w}_u$ que se comporta como un modelo que nunca hubiera sido entrenado con el conjunto de olvido, $\mathcal{A}(\mathcal{D}_r)$.

Podemos obtener el modelo de referencia para un *desaprendizaje* exacto aplicando el algoritmo $\mathcal{U} = \mathcal{U}(\mathcal{A}(\mathcal{D}_r), \mathcal{D}_r)$ sin emplear el conjunto de olvido. En este caso se garantiza que este modelo no contiene información sobre los datos cuya influencia se quiere eliminar. En caso contrario, si $\mathcal{U} = \mathcal{U}(\mathcal{A}(\mathcal{D}), \mathcal{D}_r, \mathcal{D}_u)$, es necesario un método de verificación que verifique que el nuevo modelo se comporta como un modelo re-entrenado, y no ha sido influenciado por $\mathcal{D}_u$. Discutiremos más adelante el método de verificación que emplearemos en ese trabajo.

El objetivo del MU no es solo reproducir un modelo que se comporta como $\mathcal{A}(\mathcal{D}_r)$, sino hacerlo en el menor tiempo posible. Esto se debe a que el coste computacional y el tiempo asociado a re-entrenar un modelo desde cero puede ser enorme.

Medir el tiempo experimentalmente es difícil porque tanto el hardware como el software introducen variabilidad en las mediciones. Para evitar este problema, los autores de la referencia [6] midieron el tiempo de *desaprendizaje* indirectamente a través del número de muestras necesarias para re-entrenar. En un experimento controlado, ellos validaron la relación lineal entre el número de muestras de re-entrenamiento y el tiempo de entrenamiento de un modelo. Para los conjuntos de datos SVHN y Purchase, los resultados mostraron que el número de muestras para re-entrenar es proporcional al tiempo de re-entrenamiento (ver la Figura 2.5). Además, cuando tratamos con redes neuronales, es evidente que el tiempo de entrenamiento también es proporcional al número de épocas que se llevan a cabo durante el entrenamiento. Por tanto, muchos algoritmos de MU se centran en reducir el número de datos que se emplean para entrenar un nuevo modelo partiendo del modelo entrenado con todos los datos, y también en reducir el número de épocas necesarias para obtener el nuevo modelo.

A continuación, vamos a discutir dos estrategias que persiguen estos objetivos.

2.2.1. Reorganización de datos

La reorganización de datos se refiere a la técnica mediante la cual se olvidan ciertos datos más fácilmente reorganizando el conjunto de datos de entrenamiento. Esto engloba

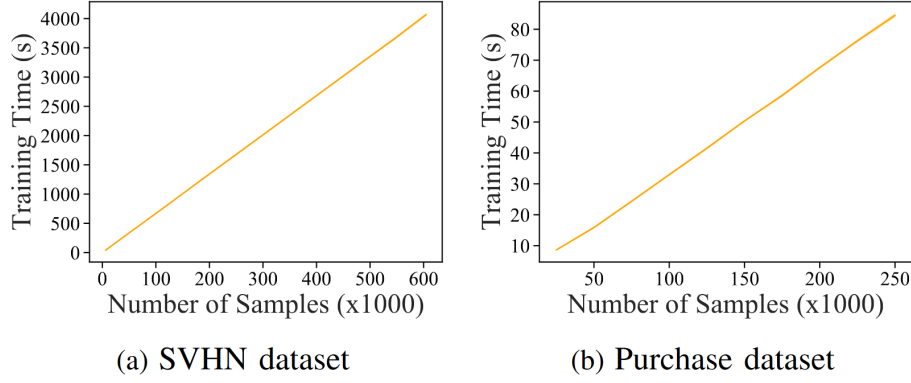


Figura 2.5: Los autores de [6] validaron la relación lineal entre el tiempo de entrenamiento y el número de muestras empleadas. Las mediciones se obtuvieron en incrementos del 10 % del tamaño del conjunto de datos, utilizando los conjuntos de datos SVHN y Purchase.

métodos de reorganización diferentes, pero en este trabajo vamos a llevar a cabo una fragmentación de los datos.

En la fragmentación de datos, primero se segmenta el conjunto de datos de entrenamiento en varios subconjuntos y se entrenan varios submodelos utilizando cada subconjunto por separado. Esos submodelos se usan luego para agregar una predicción consensuada de manera colaborativa, es decir, $\mathcal{D} \rightarrow \mathcal{D}_1 \cup \mathcal{D}_2 \cup \dots \cup \mathcal{D}_m$, $\mathbf{w}_i = \mathcal{A}(\mathcal{D}_i)$ y el modelo se obtiene mediante $\text{Agg}(\mathbf{w}_i)$, donde $\mathcal{D}_i$, $0 < i < m$ son los subconjuntos, y $\text{Agg}(\cdot)$ es la función de agregación (ver la Figura 2.6). Cuando se requiere olvidar un conjunto de datos, se eliminan esas muestras de los subconjuntos que las contienen y luego se vuelven a entrenar únicamente los submodelos afectados. La flexibilidad de esta metodología radica en que la influencia del conjunto de datos a olvidar $\mathcal{D}_u$ se limita a cada subconjunto después de la segmentación, en lugar de afectar a todo el conjunto de datos. Así se reduce el número de datos con los que hay que re-entrenar el modelo y, en consecuencia, reduciendo el tiempo empleado en el *desaprendizaje*.

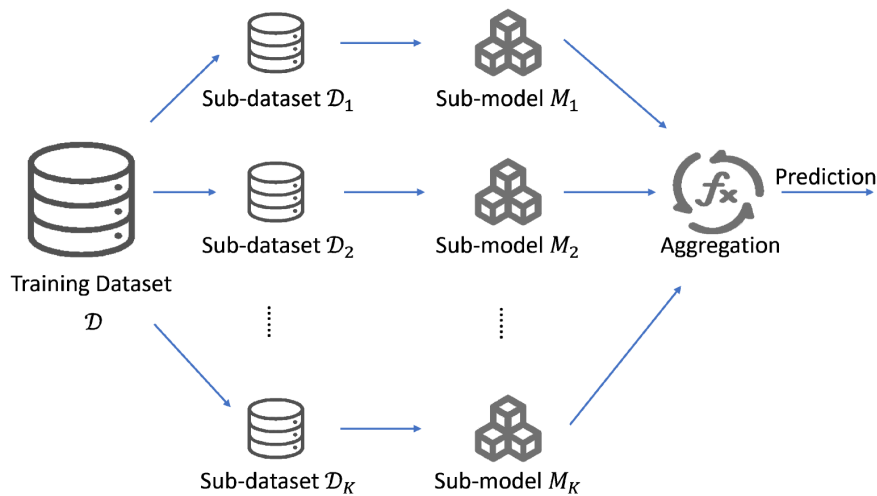


Figura 2.6: Esquema de la fragmentación de un conjunto de datos para entrenar distintos submodelos, que son agregados para formar un modelo entrenado con el conjunto completo.

La principal ventaja de este método es que el *desaprendizaje* es exacto. Dado que el submodelo que había sido entrenado con el conjunto de olvido es re-entrenado desde cero con los datos restantes, se verifica por definición que la influencia de los datos que se querían eliminar ha sido borrada.

La principal desventaja de la fragmentación es que los submodelos son entrenados con un menor volumen de datos que un único modelo entrenado con todo el conjunto de datos. Esto puede perjudicar seriamente a la precisión del modelo. Además, si el conjunto de olvido es muy grande, puede ocurrir que muchos de los submodelos hayan sido entrenados con muestras de este conjunto y por tanto haya que re-entrenar todos ellos, aumentando el tiempo empleado. Sin embargo, ciertos estudios sugieren que el número de datos que requieren ser olvidados suelen ser del orden de 1 entre un millón [7], con lo que este aspecto no supone un problema en la práctica.

2.2.2. Manipulación del modelo

En la manipulación de modelos, se busca realizar el *desaprendizaje* ajustando directamente los parámetros del modelo. Se actualizan directamente los parámetros del modelo para contrarrestar la influencia de las muestras olvidadas en el modelo, es decir, $\mathbf{w}' = \mathbf{w} + \delta$, donde $\mathbf{w}$ son los parámetros del modelo original y δ es el valor de actualización. Estos métodos se basan generalmente en la idea de calcular la influencia de las muestras en los parámetros del modelo y luego actualizar dichos parámetros para eliminar esa influencia. Como hemos mencionado en el capítulo de ML, es extremadamente difícil calcular con precisión la influencia de una muestra en los parámetros de un modelo, especialmente con modelos complejos de redes neuronales profundas. Por lo tanto, muchos métodos de MU basados en la manipulación del modelo se fundamentan en suposiciones específicas.

Muchos métodos de manipulación comienzan por introducir ruido a todos los parámetros del modelo, como se muestra a continuación:

$$\mathbf{w}' = \mathbf{w} + \mathcal{N}(0, \sigma^2), \quad (10)$$

donde $\mathcal{N}(0, \sigma^2)$ es una distribución normal con desviación típica σ . Este procedimiento puede denominarse una depuración de pesos, que implica ajustar los pesos del modelo para olvidar información, lo cual puede hacerse de forma eficiente sin comprometer significativamente el rendimiento general del modelo. Por supuesto, la precisión del modelo se verá afectada tras la depuración, pero es esperable que podrá recuperarse entrenando el modelo con el conjunto de datos retenidos en pocas épocas, reduciendo el tiempo necesario en comparación con un re-entrenamiento que parta de unos parámetros totalmente aleatorios.

La manipulación de los parámetros del modelo no es un método exacto de *desaprendizaje*, pues por la metodología empleada no se garantiza que la influencia del conjunto de datos de olvido ha sido eliminada por completo. Discutiremos a continuación el método de verificación que emplearemos para comprobar que se logra este objetivo tras llevar a cabo la manipulación.

2.2.3. Verificación del olvido: *Membership Inference Attack*

Verificar si el método de MU garantiza que la influencia del conjunto de datos de olvido haya sido borrada no es una tarea fácil. Los proveedores de modelos pueden afirmar

que eliminan esas influencias de sus modelos, pero en realidad esto puede no ser cierto. Eliminar una pequeña porción de muestras causa un efecto poco significativo en el modelo. Además, incluso si las muestras olvidadas han sido realmente eliminadas, el modelo aún tiene una gran probabilidad de hacer una predicción correcta, ya que otras muestras similares permiten que el modelo haga predicciones generalizadas.

Hoy en día, los métodos más empleados para la verificación están basados en ataques. Un propósito esencial del MU es reducir las filtraciones de información sensible causadas por el sobreajuste del modelo, y este es el mismo principio que emplean los ataques conocidos como *Membership Inference Attacks* (MIAs), que tratan de determinar si un conjunto de datos ha sido empleado para entrenar un modelo, sin conocer el conjunto de entrenamiento [8]. Por lo tanto, los MIAs pueden servir como una prueba de que la influencia del conjunto de olvido ha sido eliminada de un modelo.

En nuestro caso, vamos a emplear un MIA sencillo basado en un modelo de regresión logística que predice si un dato ha sido empleado o no para entrenar el modelo que queremos verificar.

La regresión logística se entrena a partir de datos que pertenecen a dos clases: datos con los que el modelo ha sido entrenado ($y = 1$), y datos que no han sido empleados para entrenar al modelo ($y = 0$). Como debido al sobreajuste de los modelos, estos tienen mayor precisión (o menor coste) para los datos con los que han sido entrenados, puede emplearse el valor del coste de cada muestra para predecir la clase a la que pertenece.

Si entrenamos el modelo del MIA con el conjunto de entrenamiento ($y = 1$) y con el conjunto de prueba ($y = 0$), es de esperar que tenga una precisión mayor que el 50 % para determinar si una muestra se ha empleado o no para entrenar el modelo que se ataca. Sin embargo, si entrenamos el MIA con el conjunto de olvido ($y = 1$) una vez llevado a cabo el *desaprendizaje*, cabe esperar que el ataque falle porque la influencia de este conjunto en el modelo atacado debe haber sido borrada, y la precisión del MIA será del 50 %, lo cual demuestra que es incapaz de diferenciar las muestras que se han empleado en el entrenamiento de las que no lo han sido.

En resumen, para verificar que el *desaprendizaje* de un conjunto de olvido de un modelo ha sido efectivo, vamos a entrenar un MIA con ese conjunto de olvido y con un conjunto de prueba que no se ha empleado nunca para entrenar el modelo. Si el MIA es ineficaz y no logra diferenciar ambos conjuntos —es decir, si tiene una precisión del 50 %—, entonces concluimos que la influencia del conjunto de olvido en el modelo ha sido eliminada completamente.

3. Resultados y discusión

En esta sección vamos a presentar el desarrollo de las técnicas de *Machine Unlearning* que se han llevado a cabo con una red neuronal profunda como modelo de estudio. Se han implementado ambas estrategias descritas en la sección anterior. Primero se implementa la fragmentación de los datos, que perjudicará la precisión del modelo. Se estudiarán entonces estrategias de manipulación del modelo y una combinación de las dos para obtener un método de *desaprendizaje* que cumpla los objetivos fundamentales: eliminar la influencia de un conjunto de olvido en el modelo sin dañar la precisión, en el menor tiempo posible.

Antes de discutir estos métodos, presentamos el conjunto de datos y el modelo de estudio con el que trabajaremos.

3.1. Entrenamiento y evaluación del modelo de estudio

3.1.1. Conjunto de datos: CIFAR-10

El conjunto de datos CIFAR-10 es ampliamente utilizado en la investigación de *machine learning* para la clasificación de imágenes. Contiene 60000 imágenes de 10 clases diferentes, con 6000 imágenes de cada clase [9].

Cada imagen esta compuesta por 32×32 píxeles de colores. El color de un píxel esta codificado mediante 3 valores donde, cada uno representa la intensidad de los colores primarios mediante un número entero en la escala del 0 al 255.

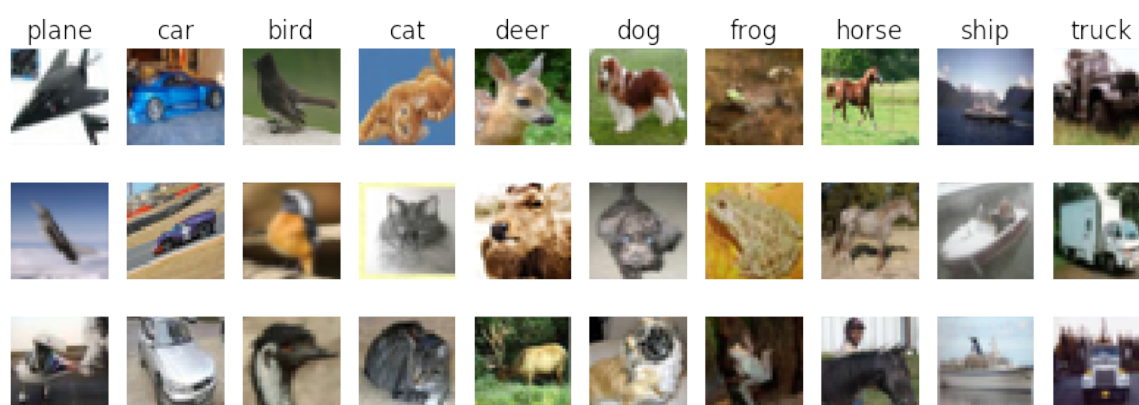


Figura 3.1: Ejemplos de las 10 clases del *dataset* CIFAR-10: avión, coche, pájaro, gato, ciervo, perro, rana, caballo, barco y camión.

Para entrenar el modelo haremos uso de un conjunto de entrenamiento con 50000 de los 60000 datos de CIFAR-10, escogidos al azar. De los 10000 datos restantes, 8000 de ellos formarán el conjunto de validación, y 2000 el conjunto de prueba.

3.1.2. Modelo de estudio: ResNet18

La red neuronal que vamos a entrenar es una red neuronal convolucional. El bloque de construcción más importante de una CNN es la capa de convolución: las neuronas en la primera capa de convolución no están conectadas a cada píxel de la imagen de entrada, sino solo a unos pocos píxeles (ver la Figura 3.2). A su vez, cada neurona en la segunda capa de convolución está conectada solo a las neuronas ubicadas dentro de un pequeño rectángulo

en la primera capa. Esta arquitectura permite que la red se concentre en características pequeñas y de bajo nivel en la primera capa oculta, y luego las agrupe en características más grandes y de mayor nivel en la siguiente capa oculta, y así sucesivamente. Esta estructura jerárquica es común en imágenes del mundo real, y es una de las razones por las que las CNNs funcionan tan bien para el reconocimiento de imágenes.

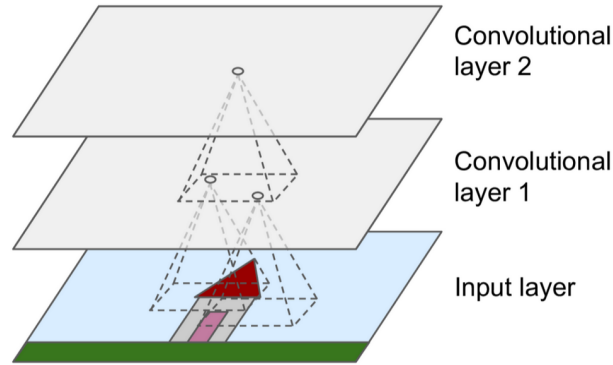


Figura 3.2: Capas convolucionales de una CNN [10].

Además de capas de convolución, las CNNs incorporan capas de *pooling*. Su objetivo es reducir los datos de entrada para disminuir la carga computacional, el uso de memoria y el número de parámetros. Al igual que en las capas de convolución, cada neurona en una capa de *pooling* está conectada a las salidas de un número limitado de neuronas en la capa anterior. Sin embargo, una neurona de *pooling* no tiene pesos; lo único que hace es agregar las entradas usando una función como el máximo o el promedio.

En concreto, vamos a entrenar un modelo con la arquitectura ResNet18, con 18 capas, adecuada para tareas de clasificación de imágenes de tamaño moderado como CIFAR-10. Este modelo tiene 11.186.112 parámetros en total.

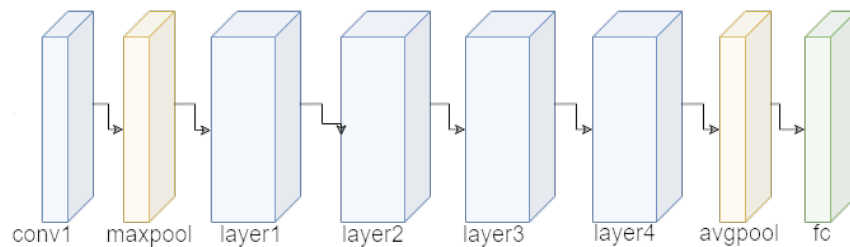


Figura 3.3: Esquema de la arquitectura del modelo ResNet18. Las capas intermedias están compuestas por varias capas convolucionales, y la última capa es una capa totalmente conectada.

3.1.3. Entrenamiento y evaluación de la precisión

Implementamos el modelo en Python, con las bibliotecas de TensorFlow y Keras. Los cálculos se realizan con una GPU GeForce RTX 2060 de NVIDIA¹.

¹Cuenta con 1920 núcleos CUDA, 240 Tensor Cores que pueden ofrecer 52 teraflops de potencia en tareas de deep learning.

A continuación se detallan los valores para los hiperparámetros en el entrenamiento del modelo. El tamaño de los lotes de datos con los que se realiza la retropropagación es de 128 muestras. La función de activación empleada por las capas es la función ReLU (ecuación (9)), y la retropropagación se realiza mediante el algoritmo de optimización Adam [11]. Por último, la función de coste es la entropía cruzada:

$$EC = - \sum_{i=1}^C t_i \log(p_i), \quad (11)$$

donde C es el número de clases, t_i los valores reales de las muestras para la clase i -ésima y p_i las predicciones dadas por el modelo para la clase i -ésima.

El modelo se entrena un total de 20 épocas, y medimos los valores del coste y la precisión en el conjunto de entrenamiento tras cada una de ellas. Medimos además la precisión en el conjunto de validación, que da realmente una medida de lo bien que funcionará el modelo clasificando imágenes que no haya visto nunca. Estos valores se representan en la Figura 3.4.

La precisión se calcula utilizando la métrica *sparse categorical accuracy* de Keras. Esta métrica evalúa el porcentaje de etiquetas predichas correctamente por el modelo comparando las etiquetas verdaderas con las predicciones del modelo. Formalmente, se define como el número de predicciones correctas dividido por el número total de muestras.

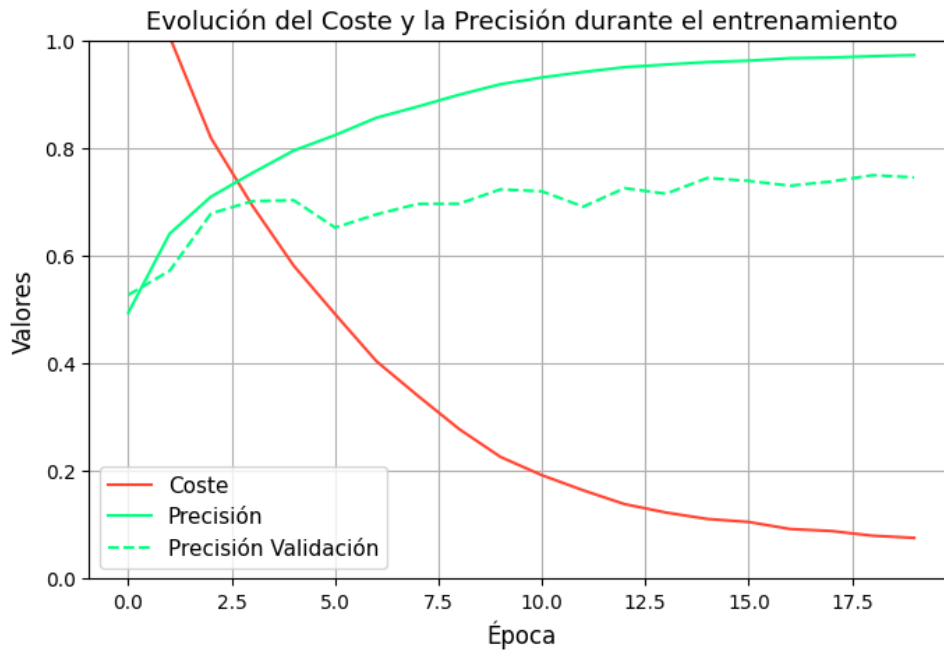


Figura 3.4: Evolución del coste, la precisión y la precisión de validación del modelo durante el entrenamiento. Se entrena un total de 20 épocas (0-19).

Tras 20 épocas, el modelo termina con una precisión del 97,4% en el conjunto de entrenamiento, y 74,6% en el conjunto de validación. Es evidente que el modelo presenta un gran sobreajuste, pero nuestro objetivo es eliminar la influencia de un conjunto de datos mediante *Machine Unlearning*. Por tanto, este modelo presenta un banco de pruebas adecuado y podremos verificar que el procedimiento de MU ha sido exitoso mediante un MIA. Estudiamos su vulnerabilidad a continuación.

3.1.4. Vulnerabilidad a un *Membership Inference Attack*

En la sección (2.2.3) ya se han introducido los MIAs como métodos de verificación de que un conjunto de olvido ha sido eliminado de un modelo. En esta sección entrenamos un MIA simple basado en una regresión logística, entrenado a partir de los costes de un conjunto de olvido que contiene el 10 % de los datos de entrenamiento (5000 imágenes), y los costes del conjunto de prueba.

Debido al sobreajuste del modelo entrenado con todos los datos de entrenamiento, la precisión en el conjunto de entrenamiento es muy elevada, y a su vez los costes son mucho menores, en comparación con la precisión y los costes en el conjunto de prueba. Se ha representado la distribución de los costes de ambos conjuntos en el primer gráfico de la Figura 3.5. Esta diferencia permite al MIA alcanzar una puntuación del 62 % a la hora de diferenciar los datos del conjunto de olvido y el conjunto de prueba.

Sin embargo, si calculamos los costes de ambos conjuntos una vez el modelo ha sido entrenado de nuevo, excluyendo el conjunto de olvido, observamos que la distribución de costes de los conjuntos de olvido y prueba son muy similares (segundo gráfico de la Figura 3.5). En este caso, el MIA tiene la peor puntuación que puede tener, un 49 % ($\approx 50\%$).

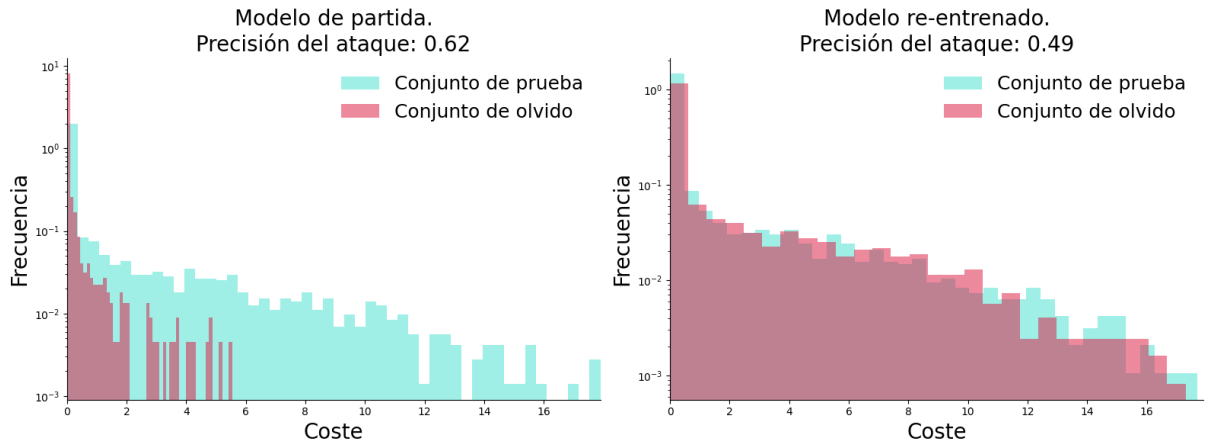


Figura 3.5: Distribución de costes de inferencia, comparando muestras de entrenamiento y muestras de prueba.

Por lo tanto, concluimos que los métodos de MU que llevemos a cabo con este modelo pueden ser verificados con un MIA sencillo basado en una regresión logística. El objetivo es que el modelo resultante del *desaprendizaje* tenga una puntuación MIA del 50 %.

3.2. Fragmentación de datos

3.2.1. Entrenamiento aislado de los fragmentos

Comenzamos estudiando cómo afecta la fragmentación de los datos a la precisión del modelo. Las 50000 imágenes de las que se compone el conjunto de entrenamiento se dividen en subconjuntos de igual tamaño, escogiendo los subconjuntos de forma aleatoria. A partir de los distintos subconjuntos se entrenan submodelos con las mismas características que el modelo base que hemos entrenado previamente.

A continuación, los distintos submodelos entrenados con los subconjuntos de los datos son agregados para formar el modelo completo. La estrategia seguida para agregar los submodelos es una votación donde cada submodelo contribuye equitativamente al resultado final a través de una simple votación por mayoría en cada una de las 10 clases para las que el modelo hace una predicción.

En los conjuntos de datos no equilibrados donde los modelos asignen altas puntuaciones a múltiples clases en lugar de a una sola clase, la agregación por votación mayoritaria pierde información sobre las clases menos relevantes y no sería adecuada. En nuestro caso, el conjunto de datos CIFAR-10 no contiene imágenes que puedan pertenecer a varias clases o sean ambiguas, por tanto, esperamos que este método de agregar todos los submodelos sea eficiente.

Los resultados se muestran en la Figura 3.6.

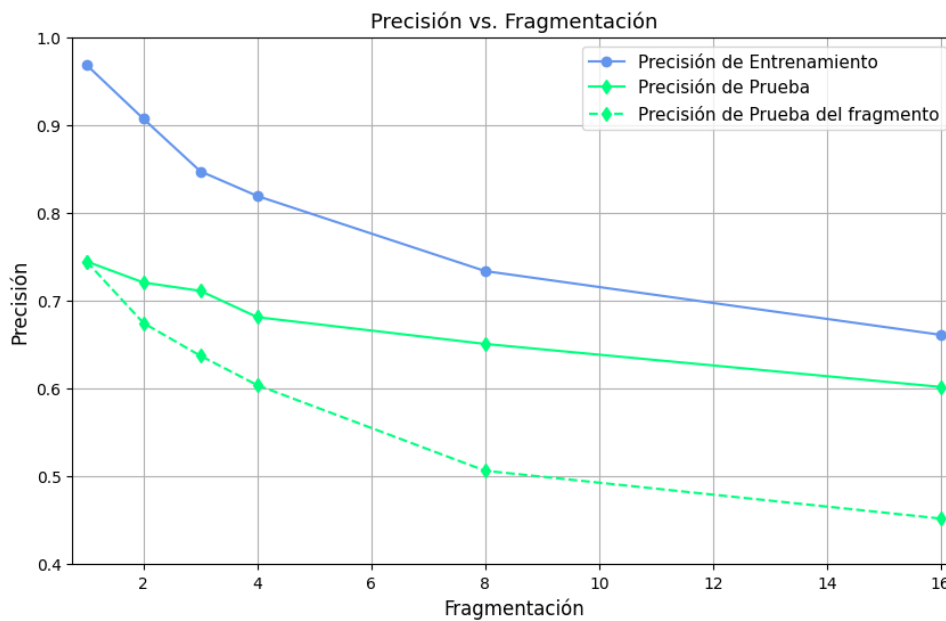


Figura 3.6: Precisión del modelo obtenido mediante fragmentación de datos en el conjunto de entrenamiento y el conjunto de prueba. También se muestra la precisión de uno de los submodelos en el conjunto de prueba. La fragmentación indica el número de fragmentos de datos y de submodelos que se han empleado.

Como esperábamos, la precisión del modelo completo se deteriora cuanto más dividimos el conjunto de datos y más submodelos agreguemos para formarlo. Esto se debe a que los submodelos entrenados únicamente con una fracción pequeña de los datos de entrenamiento no logran alcanzar la precisión del 74.6 % que alcanzaba un único modelo entrenado con todo el conjunto. Esto pone de manifiesto la importancia de los datos a la hora de entrenar un modelo de ML. Casi siempre, una mayor cantidad y calidad de los datos es más importante que emplear un modelo más potente.

Obsérvese además que, aunque un submodelo entrenado con uno de los 16 fragmentos del conjunto de entrenamiento alcance únicamente un 45 % de precisión, el modelo completo que incorpora los 16 submodelos alcanza un 60 % de precisión. Esto se debe a que se ha seguido una metodología de clasificación mediante agregación de modelos individuales.

Estas metodologías permiten que el modelo agregado obtenga un rendimiento predictivo mejor que el que podría obtenerse con cualquiera de los modelos constituyentes por sí solos. En este caso, la diferencia entre los submodelos es únicamente el conjunto de datos que se ha empleado para entrenar cada uno, pero esto introduce suficiente variedad como para mejorar significativamente el rendimiento del modelo que agrega todos ellos. Sin embargo, la mejora obtenida no es suficiente como para mantener la precisión del modelo único entrenado con todos los datos.

Recordamos que este método de crear un modelo agregado a partir de varios submodelos permite eliminar fácilmente la influencia de un conjunto de olvido, re-entrenando únicamente los submodelos afectados. Este método de MU es exacto y no es necesario verificarlo con un MIA.

3.2.2. Entrenamiento no aislado de los fragmentos

Si queremos evitar el deterioro de la precisión de nuestro modelo, una estrategia es abandonar el *desaprendizaje* exacto, es decir, dividir el modelo en distintos submodelos pero entrenarlos de forma no aislada. Si todos los datos de entrenamiento pueden utilizarse para estimar los parámetros de todos los submodelos en cierta medida, cabe esperar que el modelo generalice mejor las predicciones a nuevos datos y su rendimiento sea superior.

En el apartado anterior todos los submodelos se entrenan en 20 épocas aisladas, es decir, empleando únicamente un subconjunto de datos. Si entrenamos todos los submodelos un número N de épocas con el conjunto de entrenamiento completo (épocas no aisladas), y terminamos con $(20 - N)$ épocas con el subconjunto de entrenamiento específico, cabe esperar que el rendimiento de los modelos mejore. Sin embargo, el conjunto de olvido tendrá cierta influencia en todos los submodelos y, por tanto, tendrá que verificarse por medio de un MIA si el re-entrenamiento de estos submodelos con el conjunto de datos específico que excluye el conjunto de olvido elimina suficientemente esta influencia.

Aplicamos esta metodología en el caso en el que el conjunto de entrenamiento se divide en 8 fragmentos. En la Figura 3.7 se muestran la precisión en el conjunto de entrenamiento y en el conjunto de prueba del modelo completo. Después se re-entrena un submodelo eliminando un conjunto de olvido que contiene el 10 % del conjunto con el que se entrena, y se muestran de nuevo la precisión de entrenamiento y de prueba, así como la puntuación del MIA del modelo re-entrenado completo.

Con 0 épocas no aisladas tenemos el modelo del apartado anterior, con 8 fragmentos. La precisión de este modelo en el conjunto de prueba es de un 65 %. Observamos que a medida que introducimos un mayor número de épocas no aisladas en el entrenamiento, la precisión aumenta significativamente y alcanza un 78 % con 8 épocas no aisladas o más. Este es un rendimiento mayor al obtenido por el modelo único entrenado con todo el conjunto de entrenamiento (equivalente a 20 épocas no aisladas), debido de nuevo al efecto del aprendizaje por agregación de modelos individuales.

Por otra parte, la puntuación del MIA, que comienza siendo del 49 %, verificando que el método de *desaprendizaje* exacto funciona, empeora a medida que introducimos épocas no aisladas en el entrenamiento. Luego, aunque la precisión del modelo mejore mediante este método, es inadmisibles usarlo como un método de MU pues retiene demasiada información del conjunto de olvido en el modelo. A continuación, vamos a estudiar

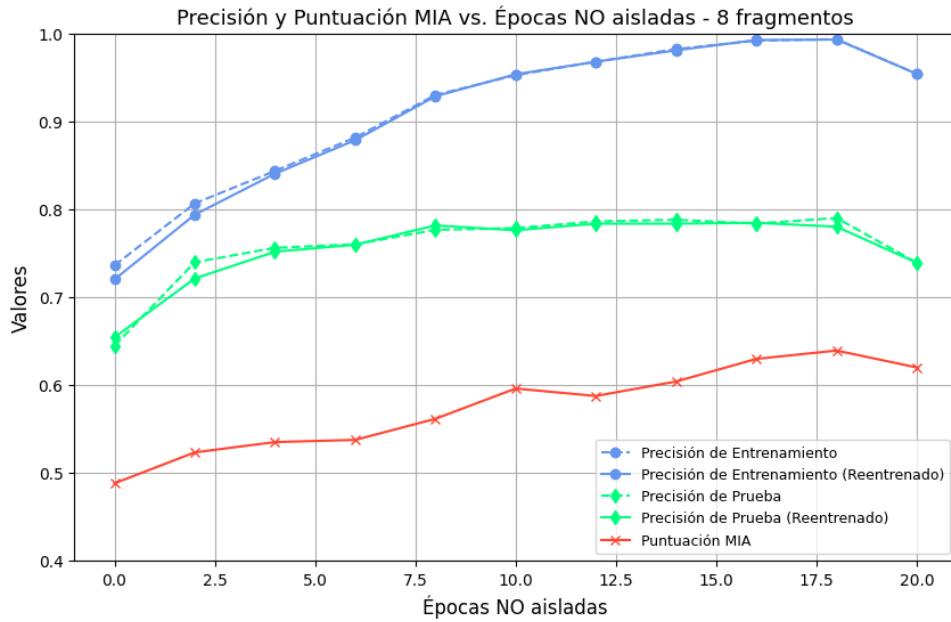


Figura 3.7: Precisión y puntuación de ataques de inferencia en entrenamiento no aislado.

técnicas de manipulación de modelos, que quizás puedan ser implementadas junto con la fragmentación para mejorar el rendimiento sin retener tanta información del conjunto de olvido.

3.3. Manipulación del modelo

Vamos a tratar de eliminar la información del conjunto de olvido del modelo mediante técnicas de manipulación directa del modelo. Estas técnicas tratan de mantener el rendimiento del modelo lo más alto posible, y a la vez reducir el tiempo necesario para ajustar el modelo.

3.3.1. Reinicialización de capas

El primer método que hemos probado se inspira en una solución de la competición de Kaggle *NeurIPS 2023 - Machine Unlearning*, desarrollada por Efstathios Karypidis y su equipo [12]. Estos investigadores restablecen las capas primera y última del modelo original, y realizan un ajuste fino posterior. Afirman que estas dos capas se eligen porque la primera capa influye significativamente en el resto de las capas del modelo y la última capa determina la distribución final de salida del modelo. Con este paso de restablecimiento, se permite que el modelo se desvíe de su estado original.

Además, sus observaciones empíricas de experimentos locales en el conjunto de datos CIFAR-10 mostraron que estas dos capas exhibían una similitud coseno negativa entre los pesos de las capas del modelo ResNet18 entrenado en el conjunto completo de entrenamiento y los modelos entrenados desde cero en el conjunto de retención (ver la Figura 3.8).

La similitud coseno entre los pesos de las mismas capas de dos modelos diferentes, $\mathbf{w}_1$ y $\mathbf{w}_2$, se calcula como:

$$S_C(\mathbf{w}_1, \mathbf{w}_2) = \frac{\mathbf{w}_1 \cdot \mathbf{w}_2}{\|\mathbf{w}_1\| \|\mathbf{w}_2\|}. \quad (12)$$

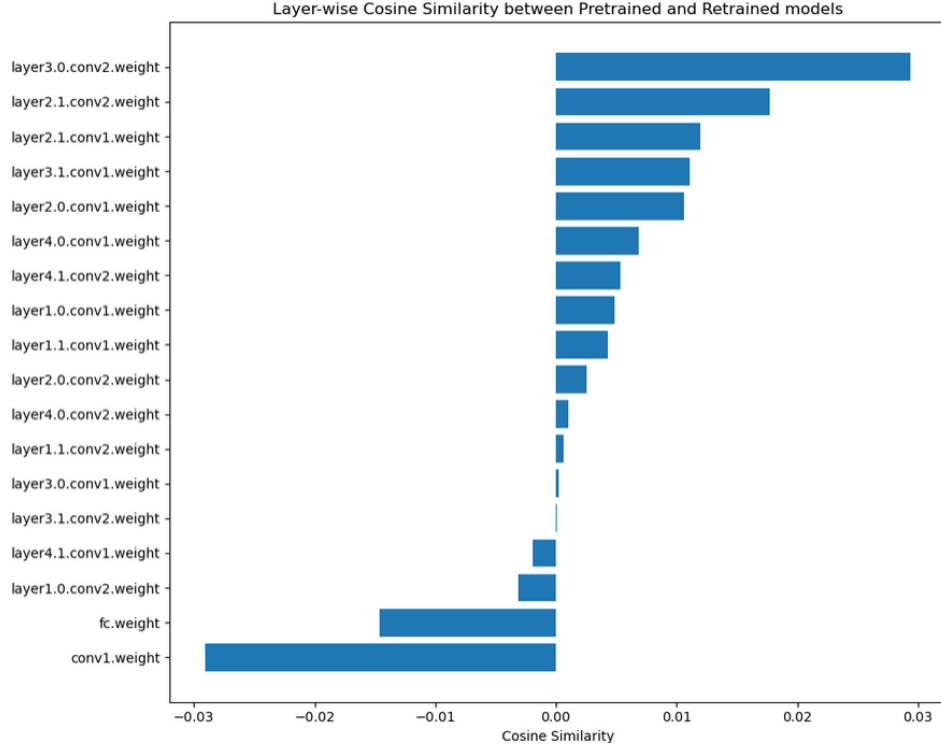


Figura 3.8: Similitud coseno entre las capas de un modelo completo y un modelo re-entrenado. *fc* (*fully connected*) corresponde a la última capa del modelo y *conv1* es la primera capa de la red convolucional.

$S_C(\mathbf{w}_1, \mathbf{w}_2)$ da una medida de lo similares que son dos capas en el espacio de parámetros. Toma valores entre -1 y 1, donde el -1 significa que las capas son totalmente opuestas, y el 1 implica que son exactamente iguales.

Para tratar de reproducir las similitudes entre las capas, se han entrenado un total de 21 modelos con el conjunto de entrenamiento, y 12 modelos re-entrenados únicamente con un conjunto de retención que contiene el 90 % del conjunto de entrenamiento. Con ellos se ha medido la similitud entre las capas de 211 pares distintos de modelos completos, y 276 pares de un modelo completo y un modelo re-entrenado. Los resultados se muestran en la Figura 3.9.

Notamos que los resultados que hemos obtenido son muy diferentes al análisis de Karypidis. Destacamos que dos modelos entrenados con el mismo conjunto completo de datos no tienen una similitud mucho mayor que la que hay entre un modelo completo y uno re-entrenado. Esto se debe a la aleatoriedad del proceso de entrenamiento, que introduce variaciones en los parámetros del modelo que se hacen más notables cuando las capas tienen muchos parámetros. Por esta razón la capa convolucional 4.0.2 de la arquitectura ResNet18, entre otras, que tiene 2.359.296 parámetros, tiene una similitud coseno tan baja.

Con respecto a la primera y la última capa del modelo, concluimos que su similitud coseno no es negativa. Sí es cierto que en algunos casos puede verse este comportamiento, pero eso se debe a que la similitud de estas capas tiene una varianza mucho mayor que la similitud del resto de capas. Esto se debe probablemente a que tienen el menor número de parámetros (9.408 y 5.130). Por tanto, no es evidente que la reinicialización de estas capas pueda tener por sí sola una gran influencia en su rendimiento tras el ajuste fino.

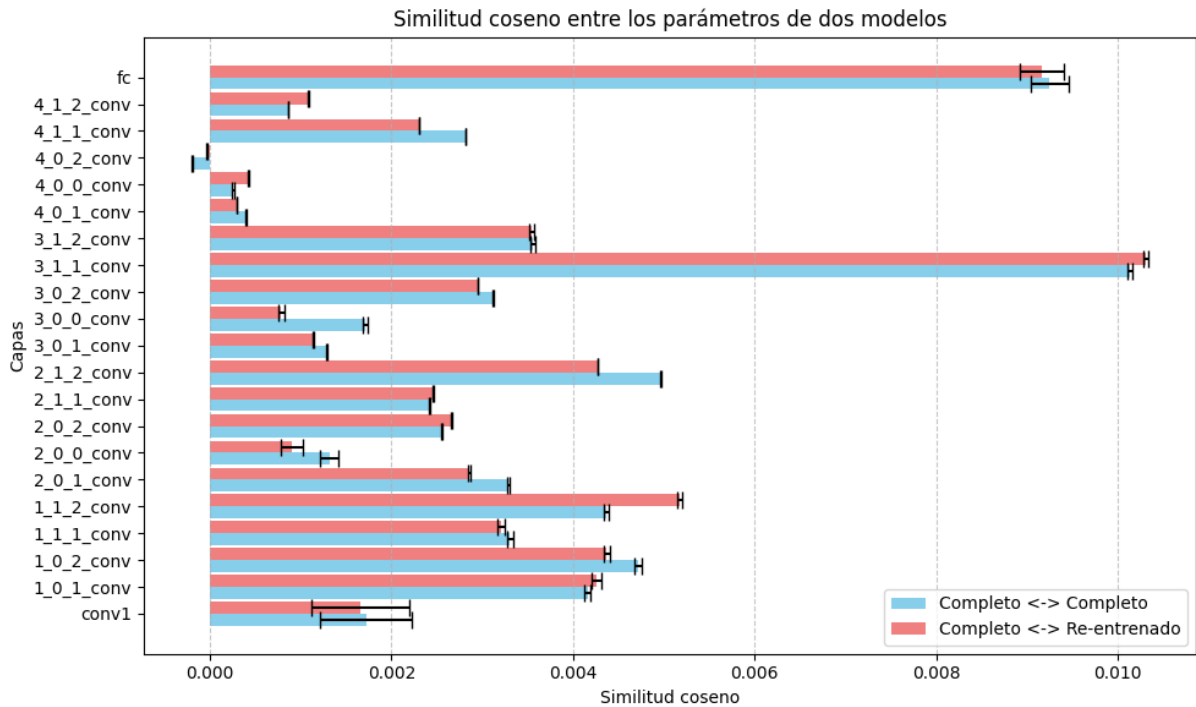


Figura 3.9: Similitud coseno entre las capas de un modelo completo y un modelo re-entrenado. Las capas están ordenadas de abajo a arriba en el orden en el que los datos las recorren.

Para comprobarlo, realizamos un ajuste fino que consiste simplemente en entrenar el modelo, cuyas primera y última capa han sido reinicializadas, durante 5 épocas. El ajuste se realiza con el conjunto de retención. Los parámetros de rendimiento durante el entrenamiento se muestran en la Figura 3.10.

Observamos que la reinicialización de la primera y última capa estropea completamente el modelo, que tiene entonces una precisión del 10 %, totalmente aleatorio para 10 clases. Sin embargo, con una sola época de entrenamiento recupera casi por completo la precisión en el conjunto de validación, alcanzando un 70 %. Cuando termina el ajuste fino, la precisión de validación es del 73,1 %. Por otro lado, se ha calculado también la precisión del modelo ajustado en el conjunto de olvido, y esta es del 87,2 % tras el ajuste. Es significativamente mayor que la precisión de validación y, en consecuencia, la efectividad del MIA esta muy por encima del 50 %: se sitúa en el 59 %.

Si realizamos este ajuste fino sencillo sin haber reinicializado la primera y última capas del modelo, encontramos que la precisión del modelo en el conjunto de olvido también se reduce de lo que hubiera sido un 97,4 % hasta un 92 %. Luego concluimos que la reinicialización de las capas puede ayudar a eliminar en cierta medida la influencia del conjunto de olvido del modelo, pero en este caso el secreto del método de MU está principalmente en el ajuste fino posterior.

Karypidis y su equipo emplearon un ajuste fino que consiste en una fase de calentamiento rápido usando el conjunto de validación para minimizar la divergencia KL entre los modelos original y re-inicializado. Luego, afinan el modelo en el conjunto de retención combinando distintos costes. Es un ajuste complejo que no vamos a implementar. En su lugar, vamos a centrarnos en el método de manipulación del modelo introducido en la

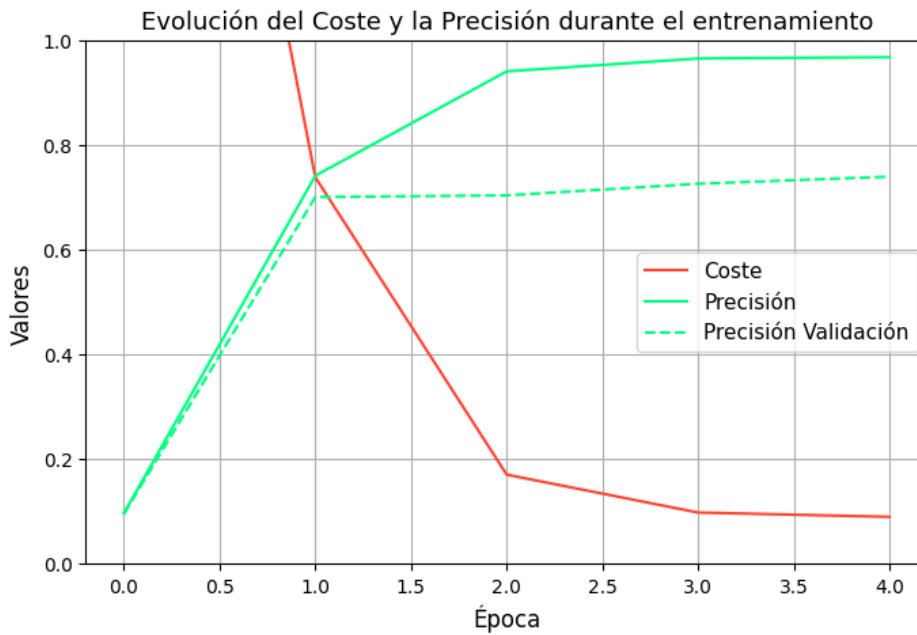


Figura 3.10: Evolución del coste, la precisión y la precisión de validación del modelo durante el ajuste fino.

sección 2.2.2: la depuración de pesos.

3.3.2. Depuración de pesos

Hemos visto que reinicializar unas pocas capas en el modelo entrenado con todos los datos permite mantener el rendimiento del modelo, pero no introduce una variación suficiente en los parámetros que permita eliminar por completo la influencia de un conjunto de olvido. La depuración de los pesos del modelo consiste en introducir ruido en todos los pesos de todas las capas del modelo. Esperamos que este método sea más eficiente olvidando la información que queremos olvidar.

Recordamos que el ruido que introducimos se controla con un parámetro: la desviación estándar σ de la distribución normal $\mathcal{N}(0, \sigma^2)$. Por ejemplo, escogemos una desviación estándar de $\sigma = 0,1$. Tras introducir el ruido en el modelo su rendimiento se deteriora enormemente, y el primer ajuste fino que vamos a probar es el mismo que en el apartado anterior: 5 épocas de entrenamiento con el conjunto de retención que contiene el 90 % de los datos de entrenamiento. Medimos los valores del coste y la precisión en los conjuntos de retención y validación tras cada época. Los valores se representan en la Figura 3.11.

Observamos que el ruido aleatorio introducido mediante $\mathcal{N}(0, \sigma^2)$ hace que el modelo parta de una situación inicial con una precisión del 15 %. Sin embargo, el modelo retiene suficiente información para alcanzar una precisión del 70 % en una única época. Este es el potencial que tiene la técnica de depuración de pesos como método de *Machine Unlearning*, pues el re-entrenamiento con los datos de retención proporciona rápidamente un rendimiento elevado.

Por supuesto, tenemos que cerciorarnos de que se ha eliminado la influencia del conjunto de olvido del modelo. Para determinar cuál es el valor óptimo del ruido que debemos introducir en el modelo, introducimos distintos niveles de ruido ($\sigma \in [0, 0,175]$) y me-

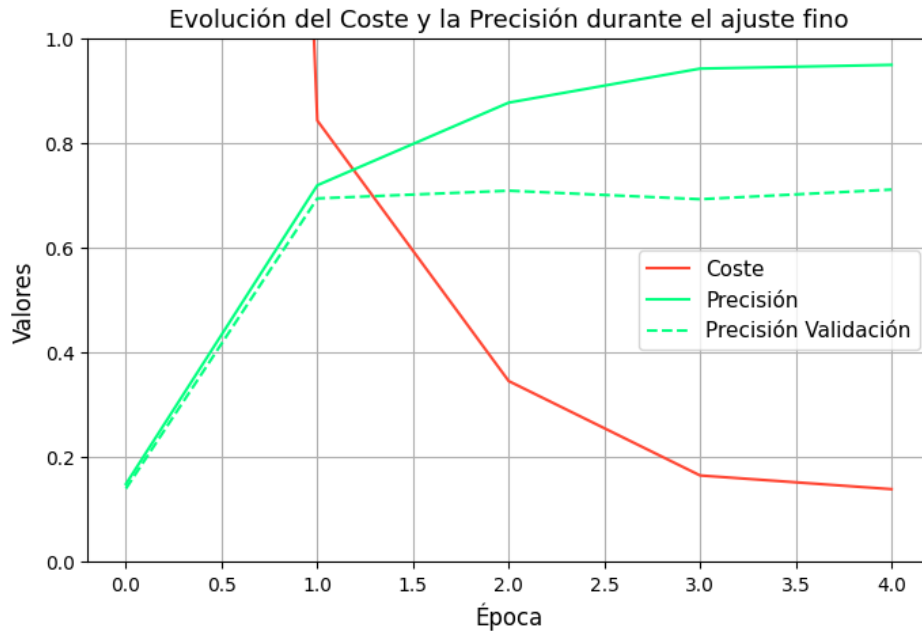


Figura 3.11: Evolución del coste, la precisión y la precisión de validación del modelo durante el ajuste fino, tras introducir ruido con $\sigma = 0,1$.

dimos la precisión del modelo que resulta del ajuste fino posterior, en los conjuntos de retención, prueba y olvido. Calculamos también la puntuación del MIA que verifica si el conjunto de olvido se ha olvidado correctamente. Debido a la aleatoriedad en el entrenamiento, los experimentos han sido realizados 3 veces, y se muestra la media y varianza de los resultados. Estos resultados se muestran en la Figura 3.12.

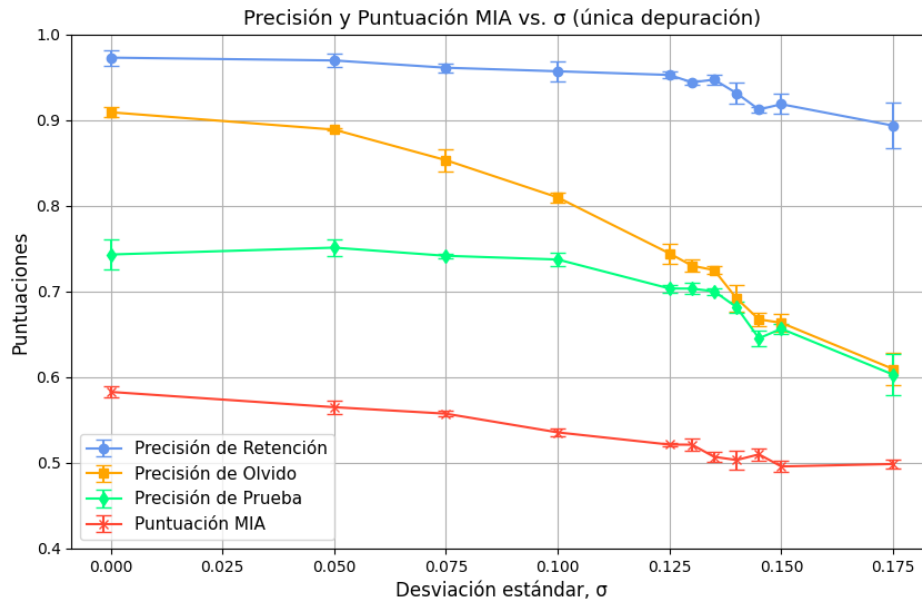


Figura 3.12: Precisiones del modelo ajustado tras una depuración con desviación estándar σ , en los conjuntos de retención, de olvido y de prueba. Se muestra también la puntuación del MIA. Todos los valores han sido calculados 3 veces, y se muestra su media y varianza.

Vamos a explicar en detalle la Figura 3.12. En un modelo entrenado con el conjunto de entrenamiento completo se ha introducido ruido gaussiano de desviación estándar σ . Posteriormente se han reajustado los parámetros con el conjunto de retención, que contiene el 90 % de los datos de entrenamiento. El 10 % restante forman el conjunto de olvido, cuya influencia en el modelo quiere eliminarse. La figura recoge la precisión del modelo reajustado para distintos valores de σ empleados, sobre los conjuntos de retención y prueba.

La precisión sobre el conjunto de retención se mantiene siempre muy por encima de la precisión sobre el conjunto de prueba, debido al sobreajuste. Si esto no fuera así, nuestro método de verificación de olvido, el MIA, fallaría. Por otro lado, recogemos la precisión del modelo reajustado en el conjunto de olvido. Esta precisión no se mantiene siempre por encima de la precisión en el conjunto de prueba, sino que ambas terminan siendo iguales cuando el ruido introducido es suficientemente elevado. El conjunto de prueba nunca se ha empleado para entrenar los modelos; por lo tanto, cuando la precisión del conjunto de olvido es igual que la de este conjunto, la puntuación del MIA se reduce hasta el 50 %. En este punto se verifica que la influencia del conjunto de olvido ha sido completamente eliminada del modelo reajustado.

El entrenamiento de los modelos da resultados algo diferentes cada vez, pero podemos estimar que el nivel de ruido que logra eliminar la influencia del conjunto de olvido en el modelo es de $\sigma \approx 0,135$. En ese caso, el rendimiento del modelo no se mantiene cerca del 75 %, sino que baja en casi 10 puntos. Por lo tanto, la técnica de la depuración de pesos, al igual que la fragmentación de los datos, es un método de MU que permite re-entrenar el modelo más rápido, empleando 5 épocas en vez de 20, a costa de la precisión del modelo.

En la siguiente sección vamos a tratar de emplear una combinación de las técnicas de fragmentación de datos y depuración de pesos. Antes de probarlo, tenemos que estudiar cuánto se deteriora la precisión del modelo re-entrenado cuando el número de datos se reduce considerablemente, como ocurrirá con los submodelos entrenados con una fracción de los datos. Realizamos de nuevo la experimentación para modelos entrenados con $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$ y $\frac{1}{16}$ de los datos de entrenamiento. Los resultados de precisión obtenidos con los distintos conjuntos para distintos valores de σ se muestran en la Figura 3.13.

Observamos una caída mucho más pronunciada de la precisión de prueba. Tiene sentido porque esta mide la capacidad del modelo de generalizar las predicciones a datos que no han sido empleados para entrenarlo, y con una menor cantidad de datos en el conjunto de retención el modelo no es capaz de recuperar esta capacidad. Además, con un menor número de datos el ajuste fino es más débil y tampoco se elimina lo suficientemente rápido la influencia de los datos del conjunto de olvido. En consecuencia, para verificar que los datos han sido olvidados hay que introducir un ruido demasiado elevado y se perjudica mucho el rendimiento del modelo.

Para intentar disminuir el ruido que insertamos en los parámetros del modelo, pero asegurar que olvidamos los datos que queremos eliminar, vamos a emplear un ajuste diferente. Si nos fijamos en las curvas de la precisión en el conjunto de olvido y en el conjunto de prueba, la primera siempre parece comenzar a caer más rápidamente que la segunda. En la Figura 3.12, por ejemplo, cuando $\sigma = 0,1$ la precisión de olvido ya ha caído en 10 puntos porcentuales, cuando la precisión de prueba todavía se mantiene casi en el 75 %.

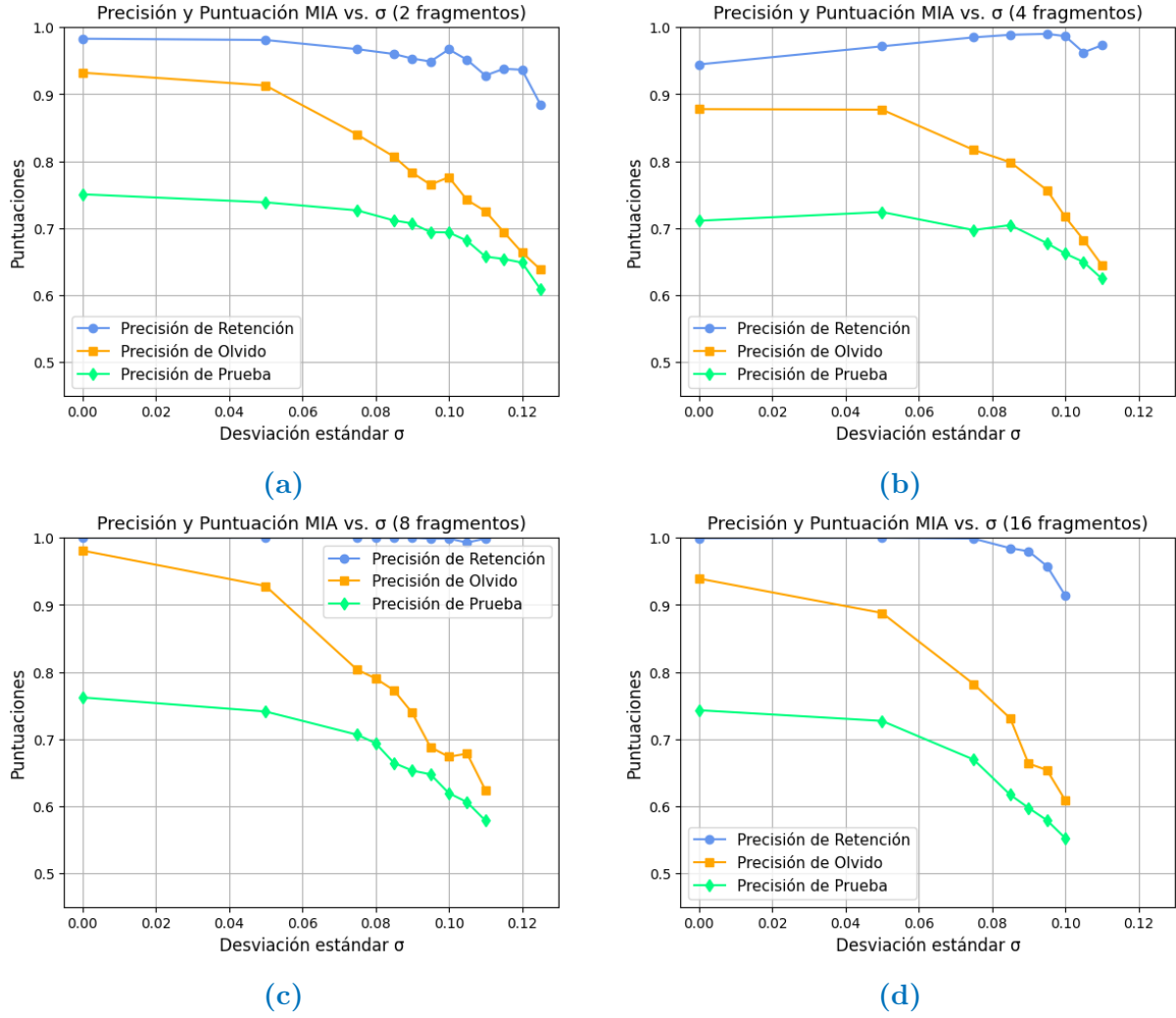


Figura 3.13: Precisiones del modelo ajustado tras una depuración con desviación estándar σ , en los conjuntos de retención, de olvido y de prueba. Modelos entrenados con (a) $\frac{1}{2}$, (b) $\frac{1}{4}$, (c) $\frac{1}{8}$ y (d) $\frac{1}{16}$ de los datos de entrenamiento.

La estrategia, por tanto, es la siguiente: en vez de introducir un ruido elevado una única vez para eliminar por completo la influencia del conjunto de olvido, y reajustar los parámetros entrenando el modelo con el conjunto de retención, vamos a introducir un ruido menor varias veces. Cada vez que introduzcamos algo de ruido se eliminará una parte de la influencia de los datos a olvidar, pero no toda. Entonces entrenamos el modelo por una sola época para recuperar el rendimiento del modelo, y repetimos la operación.

En vez de realizar un ajuste fino con 5 épocas, vamos a llevar a cabo 10 épocas de re-entrenamiento, para estar seguros de que eliminamos toda la influencia del conjunto de olvido. Entre las 10 épocas, vamos a introducir 5 depuraciones adicionales, todas con el mismo valor de σ . Estamos sacrificando algo de tiempo cuando empleamos este ajuste fino con 10 épocas, pero nuestro objetivo con la depuración es principalmente evitar el deterioro del modelo en el re-entrenamiento para contrarrestar la caída de precisión por la fragmentación de los modelos.

Los pasos del ajuste fino van a ser por lo tanto:

$$D \rightarrow 1E \rightarrow D \rightarrow 1E \rightarrow D \rightarrow 1E \rightarrow D \rightarrow 1E \rightarrow D \rightarrow 2E \rightarrow D \rightarrow 4E, \quad (13)$$

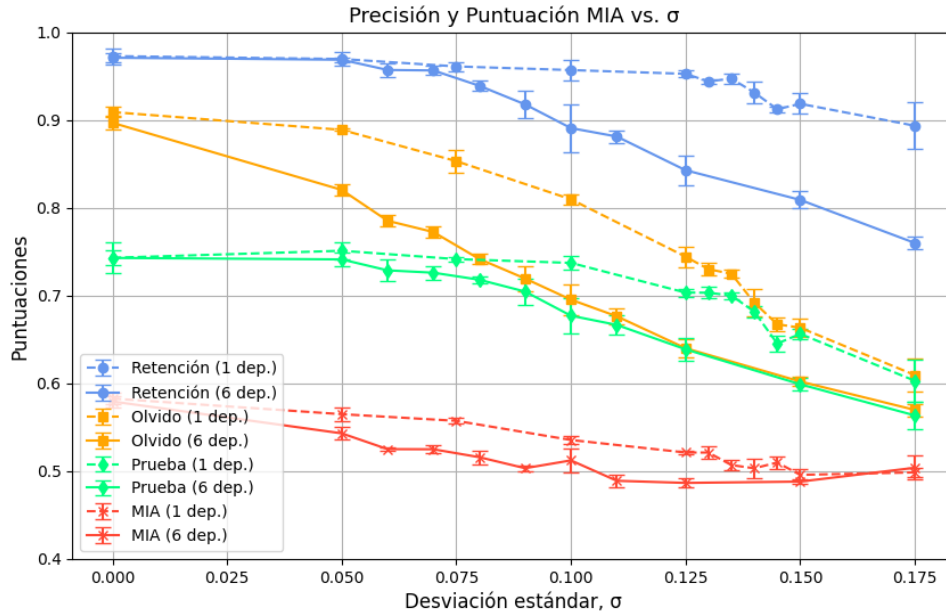


Figura 3.15: Precisiones del modelo ajustado tras 1 y tras 6 depuraciones con desviación estándar σ , en los conjuntos de retención, de olvido y de prueba. Se muestra también la puntuación del MIA. Todos los valores han sido calculados 3 veces, y se muestra su media y varianza.

modelo con una buena precisión.

La estrategia de MU es por tanto la siguiente:

1. *Entrenamiento.* Se entrena un modelo por 20 épocas con el conjunto de entrenamiento completo.
2. *Fragmentación.* Se fragmenta el conjunto de entrenamiento en N subconjuntos de igual tamaño.
3. *Depuración.* Se depuran los pesos del modelo completo, y mediante un ajuste fino con los N subconjuntos, se crean N submodelos que retienen mucha influencia de todos los datos.
4. *Agregación.* Los N submodelos se agregan mediante votación por mayoría para crear un modelo único.
5. *Desaprendizaje.* Cuando se requiere eliminar la influencia en el modelo de un conjunto de olvido, se parte desde el paso 3 con los submodelos que hayan sido entrenados con datos de este subconjunto. Se reajustan esos submodelos con los subconjuntos de retención.

Se han realizado todos estos pasos con 1, 2, 4, 8 y 16 fragmentos. El conjunto de olvido lo forman el 10 % de los datos, repartidos equitativamente en todos los subconjuntos. En los 4 primeros casos, el ajuste fino sigue los pasos descritos por la expresión (13). En el último caso, con 16 fragmentos, los últimos pasos que recogen un ajuste de 2 épocas, y una depuración seguida de un ajuste de 4 épocas, se sustituyen por un ajuste de 6 épocas seguidas. Esto mejora la precisión del modelo final, pues con los datos fragmentados en

16 subconjuntos de retención con solo 2812(3) datos, no se logra recuperar el máximo rendimiento con solo 4 épocas de entrenamiento.

En la Tabla 3.1 se muestran los valores de la desviación estándar del ruido empleado en las depuraciones de los parámetros para cada nivel de fragmentación del modelo.

Tabla 3.1: Valores de la desviación estándar σ del ruido gaussiano empleado para la depuración de pesos de los submodelos creados a partir de los N fragmentos de los datos de entrenamiento.

N	1	2	4	8	16
σ	0.9	0.85	0.8	0.7	0.6

Cuanto menor es el número de datos que se usan para entrenar cada submodelo, menor es el ruido requerido para eliminar la influencia del conjunto de olvido, una tendencia que ya hemos observado en la Figura 3.13.

Los resultados de la precisión del modelo final para los distintos números de fragmentos, y puntuación del MIA que verifica el *desaprendizaje* se muestran en la Figura 3.16.

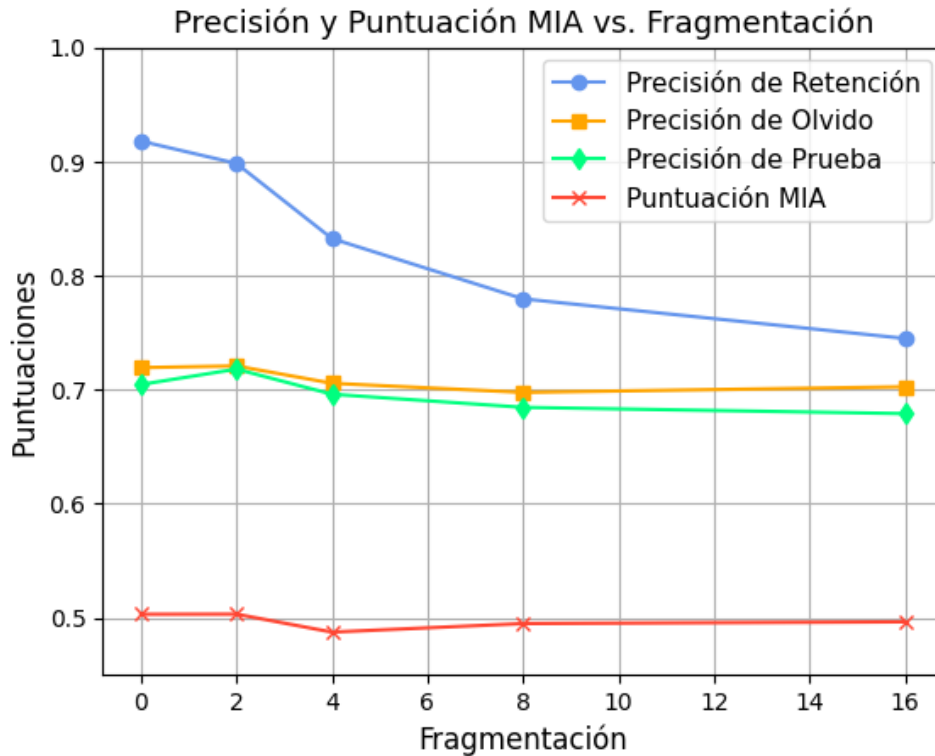


Figura 3.16: Precisiones del modelo cuyos submodelos se han ajustado mediante varias depuraciones, frente al número de submodelos que forman el modelo completo mediante agregación. Se muestra también la puntuación del MIA.

En esta figura se observa que la combinación de la fragmentación con la depuración de los pesos permite que la fragmentación del modelo en varios submodelos no perjudique

seriamente su rendimiento. A diferencia de los resultados expuestos en la Figura 3.6, donde con 16 fragmentos la precisión en el conjunto de prueba desciende hasta el 60 %, en este caso se mantiene en un 68 %. Considerando el pequeño volumen de datos que contiene cada uno de los submodelos, concluimos que la retención de información y el rendimiento obtenidos han sido satisfactorios.

Con respecto a la verificación de que la influencia del conjunto de olvido ha sido eliminada, se observa que la puntuación del MIA se mantiene en todos los casos alrededor de un 50 %, incapaz de diferenciar las muestras que fueron empleadas para entrenar el modelo de las que no.

La principal desventaja de esta metodología que permite el rápido *desaprendizaje* de un conjunto de datos, es que no se ha logrado mantener por completo el rendimiento que presentaba el modelo de partida entrenado con todos los datos de entrenamiento, bajando hasta una precisión del 74,6 %.

Por otro lado, con un total de 16 fragmentos, y un re-entrenamiento basado en la depuración de pesos que requiere un ajuste en 10 épocas, los submodelos se re-entrenan 32 veces más rápido que el modelo original. Si bien es cierto que cuando el conjunto de olvido es muy grande cabe la posibilidad de que más de un subconjunto tenga que ser re-entrenado, ya hemos mencionado previamente que en la práctica esto es poco probable. Además, los modelos que empleen un conjunto de entrenamiento con un volumen de datos mucho mayor podrán fragmentarse potencialmente en muchos más fragmentos sin dañar significativamente la precisión del modelo.

4. Conclusiones

Este trabajo demuestra que las técnicas de *Machine Unlearning* son efectivas para eliminar la influencia de datos específicos en modelos de redes neuronales profundas, aunque con ciertas limitaciones en términos de precisión y eficiencia.

La reinicialización de capas sin un ajuste fino sofisticado no ha sido exitosa eliminando la influencia de los datos. Por otro lado, la fragmentación de los datos y la depuración de los pesos logran este objetivo por sí solos. Sin embargo, la combinación de ambos métodos se presenta como la metodología más prometedora. Se ha determinado que un ajuste fino basado en múltiples depuraciones más débiles da los mejores resultados, aunque todavía hay mucho margen de mejora.

Para futuras investigaciones, cabe mencionar las siguientes líneas:

- *Optimización de técnicas de desaprendizaje.* Investigar métodos más avanzados de manipulación y fragmentación que minimicen la pérdida de precisión y mejoren la eficiencia del re-entrenamiento es probablemente todavía lo más importante. En el ámbito de la manipulación de los modelos, podrán investigarse ajustes que empleen funciones de coste y técnicas de optimización más sofisticadas para recuperar el rendimiento tras la manipulación. Con respecto a la fragmentación, quizás la mejora más evidente es el uso de un método de agregación de modelos más inteligente.
- *Aplicación a diferentes modelos y conjuntos de datos.* Es importante extender las técnicas propuestas a otros tipos de modelos y conjuntos de datos más grandes y complejos para validar su eficacia y generalización.
- *Seguridad y privacidad.* Analizar y mejorar las implicaciones de seguridad y privacidad asociadas con el MU es fundamental, empleando métodos de verificación más robustos.
- *Implementación práctica.* Finalmente, es necesario desarrollar herramientas que faciliten la integración de técnicas de MU en sistemas de aprendizaje automático utilizados en la industria.

Estas líneas de trabajo futuras permitirán avanzar en la implementación efectiva del derecho al olvido en el ámbito de la inteligencia artificial y el aprendizaje automático, contribuyendo a la creación de tecnologías más responsables y seguras.

Referencias

- [1] T. Li, E. Fosch Villaronga, and P. Kieseberg, “Humans forget, machines remember: Artificial intelligence and the right to be forgotten,” *Computer Law & Security Review*, vol. 34, p. 304, 2018. [Online]. Available: https://scholarship.law.bu.edu/faculty_scholarship/817
- [2] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, “Membership inference attacks from first principles,” 2022.
- [3] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.
- [4] F. Rosenblatt, “The perceptron - a perceiving and recognizing automaton,” Cornell Aeronautical Laboratory, Ithaca, New York, Tech. Rep. 85-460-1, January 1957.
- [5] D. E. Rumelhart and J. L. McClelland, *Learning Internal Representations by Error Propagation*, 1987, pp. 318–362.
- [6] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, “Machine unlearning,” 2020.
- [7] T. Bertram, E. Bursztein, S. Caro, H. Chao, R. C. Fegan *et al.*, “Five years of the right to be forgotten,” in *Proceedings of the Conference on Computer and Communications Security*, 2019.
- [8] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” 2017.
- [9] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Technical Report, 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [10] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems*. Sebastopol, CA: O’Reilly Media, 2019.
- [11] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2017.
- [12] E. Karypidis, V. Perifanis, C. C. Nikolaidis, N. Komodakis, and P. Efraimidis, “6th place solution for the neurips 2023 - machine unlearning competition,” 2023. [Online]. Available: <https://www.kaggle.com/competitions/neurips-2023-machine-unlearning/discussion/458740>

A. Repositorio de Código

El código fuente utilizado en este trabajo está disponible en el siguiente repositorio de GitHub:

<https://github.com/roman1618/Machine-Unlearning-with-DNNs>